

EroMergeSort Team Notebook

Manual de Referencia Completo (C++)

14 de agosto de 2026

Índice

1	Plantillas Base y Optimización I/O	4
1.1	Configuración Inicial (Template C++)	4
1.2	Lectura Eficiente (Fast I/O)	4
2	Estructuras de Datos y Búsqueda	5
2.1	Búsqueda Binaria (Algoritmo Floor)	5
2.2	Fenwick Tree (Binary Indexed Tree)	5
2.3	Segment Tree (Consultas por Rangos)	5
2.4	Segment Tree con Lazy Propagation	5
2.5	Merge Sort Tree	6
2.6	Sparse Table (RMQ Estático)	6
2.7	DSU con Unión por Tamaño (Rollback-Ready)	6
2.8	Trie (Árbol de Prefijos)	6
3	Lower Bound y Upper Bound	8
3.1	Conceptos y Uso Básico en STL	8
3.2	Búsqueda con Comparador Personalizado	8
3.3	lower_bound / upper_bound en <code>set</code> y <code>multiset</code>	8
3.4	Búsqueda Binaria Genérica sobre Predicado (Adaptativa)	8
3.5	Implementación Manual (Referencia Interna)	9
4	Algoritmos de Grafos	10
4.1	Tabla de Decisión Rápida	10
4.2	Plantilla Universal de Grafo	10
4.3	BFS (Búsqueda en Anchura)	10
4.3.1	BFS Multi-Fuente	10
4.3.2	BFS 0/1 (Deque)	10
4.4	DFS (Búsqueda en Profundidad)	11
4.4.1	DFS Iterativo (evita stack overflow)	11
4.5	Componentes Conexas y Grafos Bipartitos	11
4.6	Detección de Ciclos	11
4.6.1	Grafo NO Dirigido	11
4.6.2	Grafo Dirigido (3 colores)	11
4.7	Ordenamiento Topológico	11
4.7.1	Kahn (BFS) — Recomendado	12
4.7.2	DFS (post-orden invertido)	12
4.8	Caminos Mínimos con Pesos	12
4.8.1	Dijkstra (pesos no negativos)	12
4.8.2	Bellman-Ford (permite pesos negativos)	12
4.8.3	Floyd-Warshall (todos los pares)	12
4.9	Árbol de Expansión Mínima (MST)	12
4.9.1	Kruskal (recomendado, más fácil de codear)	12
4.9.2	Prim (mejor para grafos densos)	12
4.10	Componentes Fuertemente Conexas: Tarjan	13
4.11	Puentes y Puntos de Articulación	13
4.12	Ancestro Común Más Bajo (LCA) — Binary Lifting	13
4.13	2-SAT	13
4.14	Flujo Máximo: Edmonds-Karp	14
4.15	Grafos Implícitos en Grid/Matriz	14
4.16	Errores Comunes que Cuestan Penalizaciones	14

5	Cadenas (Strings)	15
5.1	KMP (Knuth-Morris-Pratt)	15
5.2	Z-Function	15
5.3	Hashing de Strings (Polinomial)	15
5.4	Manacher (Palíndromos)	15
5.5	Boyer-Moore (Búsqueda de Patrones)	15
6	Programación Dinámica (DP)	17
6.1	¿Cuándo Usar Programación Dinámica?	17
6.2	Plantilla General de DP (Top-Down / Memoización)	17
6.3	Fibonacci con Memoización	17
6.3.1	Versión con Arreglo	17
6.3.2	Versión con <code>unordered_map</code>	17
6.3.3	Fast Doubling con Memoización	18
6.4	Mochila 0-1 (0-1 Knapsack)	18
6.5	Longest Common Subsequence (LCS)	18
6.6	Longest Increasing Subsequence (LIS)	18
6.7	DP con Máscaras de Bits (Bitmask DP - TSP)	18
6.8	DP en Árboles (Tree DP)	18
7	Manipulación de Bits (Bitwise)	20
7.1	Operaciones Básicas y Trucos Fundamentales	20
7.2	Conteo de Bits (Popcount)	20
7.3	Suma y Resta Binaria (Bit a Bit)	20
7.4	Multiplicación Binaria (Russian Peasant / Egyptian Multiplication)	20
7.5	Enumeración de Submáscaras (Submask Enumeration)	21
7.6	Propiedades y Trucos con XOR	21
7.7	<code>std::bitset</code> : Uso y Conversión	21
7.8	Casos de Uso de Bitmasking	22
8	Matemáticas Teóricas	23
8.1	MCD (Máximo Común Divisor) y MCM (Mínimo Común Múltiplo)	23
8.2	Algoritmo de Euclides Extendido	23
8.3	Criba de Eratóstenes (Sieve)	23
8.4	Exponenciación Rápida (Fast/Modular Power)	23
8.5	Inverso Multiplicativo Modular	23
8.6	Combinatoria: Factoriales y nCr con Módulo	23
8.7	Test de Primalidad Miller-Rabin	24
8.8	Teorema Chino del Resto (CRT) — Implementación	24
8.9	Función de Möbius	24
8.10	Fórmula de Legendre (Exponente de un Primo en $N!$)	24
9	Geometría Computacional	25
9.1	Primitivas Básicas: Punto, Producto Cruz y Punto	25
9.2	Envoltente Convexa (Convex Hull - Andrew Monotone Chain)	25
9.3	Área y Punto Dentro de Polígono	25
9.4	Intersección de Segmentos	25
9.5	Punto de Intersección de Dos Rectas	25
9.6	Distancia Punto-Recta y Punto-Segmento	26
9.7	Círculos: Pertenencia e Intersección	26
9.8	Par de Puntos Más Cercano (Closest Pair)	26
9.9	Rotating Calipers: Diámetro de un Conjunto de Puntos	26
9.10	Ordenamiento Angular (Radial Sort) y Convexidad	27
9.11	Interpolación de Hermite (Cúbica)	27
9.12	Spline Cúbico Natural	27

10 Fórmulas Matemáticas Esenciales (Implementadas)	29
10.1 Sumas y Series	29
10.2 Combinatoria	29
10.3 Geometría	29
10.4 Probabilidad y Recurrencias	30
10.5 Teorema Maestro (guía de referencia, no se codea)	31
11 Casos de Estudio: Problemas Resueltos	32
11.1 Matrioshka	32

1. Plantillas Base y Optimización I/O

1.1 Configuración Inicial (Template C++)

Estructura inicial limpia con macros habituales para acelerar la escritura de código durante los concursos.

```
1 #include <bits/stdc++.h>
2 using namespace std;
3 #define fastio ios_base::sync_with_stdio(0); cin.tie(0);
4   cout.tie(0);
5 #define pb push_back
6 #define all(v) v.begin(), v.end()
7 typedef long long ll;
8 typedef pair<int, int> pii;
9 int main() {
10     fastio;
11     // Tu código aquí
12     return 0;
13 }
```

1.2 Lectura Eficiente (Fast I/O)

Complejidad de Tiempo: $\mathcal{O}(1)$ por operación de lectura amortizada.

Desactiva la sincronización pesada de flujos heredada de C y desvincula cin de cout para agilizar lecturas masivas. ****No mezclar con scanf/printf ni usar endl.****

```
1 // Incluir inmediatamente al inicio del main
2 ios_base::sync_with_stdio(false);
3 cin.tie(NULL);
```

Patrones de Entrada Comunes

```
1 void lectura_ejemplos() {
2     int n; double d; char c;
3     cin >> n >> d >> c; // Variables basicas
4     vector<int> arr(n);
5     for (int i = 0; i < n; i++) cin >> arr[i]; // Carga de
6     vectores
7     string nombre;
8     cin.ignore(); // Limpiar salto de linea remanente del
9     bufer
10    getline(cin, nombre); // Lineas completas con espacios
11    int x;
12    while (cin >> x) { /* Leer hasta Fin de Archivo (EOF)
13        */ }
14 }
```

2. Estructuras de Datos y Búsqueda

2.1 Búsqueda Binaria (Algoritmo Floor)

Complejidad de Tiempo: $\mathcal{O}(\log N)$

Complejidad de Espacio: $\mathcal{O}(1)$

Busca un valor en un rango ordenado. Si el elemento no existe, retorna de forma exacta el índice del mayor elemento inferior más cercano al buscado (*floor*). Retorna -1 si todos son mayores.

```
1 int binary_search_floor(const vector<int>& arr, int target
2 ) {
3     int low = 0, high = (int)arr.size() - 1;
4     int ans = -1;
5     while (low <= high) {
6         int mid = low + (high - low) / 2;
7         if (arr[mid] == target) return mid;
8         else if (arr[mid] < target) { ans = mid; low = mid
9             + 1; }
10        else high = mid - 1;
11    }
12    return ans;
13 }
```

2.2 Fenwick Tree (Binary Indexed Tree)

Complejidad de Tiempo:

- Inicialización: $\mathcal{O}(N)$
- Actualización puntual (Update): $\mathcal{O}(\log N)$
- Consulta por rango (Query): $\mathcal{O}(\log N)$

Complejidad de Espacio: $\mathcal{O}(N)$

Estructura eficiente para consultas de sumas acumuladas dinámicas y actualizaciones puntuales en un arreglo de tamaño N .

```
1 struct BIT {
2     int n; vector<int> tree;
3     BIT(int n) : n(n), tree(n + 1, 0) {}
4     void update(int i, int delta) {
5         for (; i <= n; i += i & -i) tree[i] += delta;
6     }
7     int query(int i) {
8         int sum = 0;
9         for (; i > 0; i -= i & -i) sum += tree[i];
10        return sum;
11    }
12    int query(int l, int r) { return query(r) - query(l -
13        1); }
14 }
```

2.3 Segment Tree (Consultas por Rangos)

Complejidad de Tiempo:

- Construcción: $\mathcal{O}(N)$
- Actualización puntual (Update): $\mathcal{O}(\log N)$
- Consulta por rango (Query): $\mathcal{O}(\log N)$

Complejidad de Espacio: $\mathcal{O}(N)$ (requiere $4N$ nodos en el peor de los casos).

Estructura de datos para procesar de manera óptima consultas asociativas en intervalos dinámicos.

```
1 struct SegTree {
2     int n; vector<int> tree;
3     SegTree(int n) : n(n), tree(4 * n, 0) {}
4     void update(int node, int start, int end, int idx, int
5         val) {
6         if (start == end) { tree[node] = val; return; }
7         int mid = (start + end) / 2;
8         if (idx <= mid) update(2 * node, start, mid, idx,
9             val);
10        else update(2 * node + 1, mid + 1, end, idx, val);
11        tree[node] = tree[2 * node] + tree[2 * node + 1];
12    }
13    int query(int node, int start, int end, int l, int r)
14    {
15        if (r < start || end < l) return 0;
16        if (l <= start && end <= r) return tree[node];
17        int mid = (start + end) / 2;
18        return query(2 * node, start, mid, l, r) + query(2
19            * node + 1, mid + 1, end, l, r);
20    }
21 }
```

2.4 Segment Tree con Lazy Propagation

Complejidad de Tiempo:

- Construcción: $\mathcal{O}(N)$
- Actualización por Rangos (Range Update): $\mathcal{O}(\log N)$
- Consulta por rango (Range Query): $\mathcal{O}(\log N)$

Complejidad de Espacio: $\mathcal{O}(N)$ (Arreglos de tamaño $4N$). Permite realizar modificaciones masivas en un rango completo de forma perezosa (*lazy*), posponiendo las actualizaciones a los nodos hijos hasta que sea estrictamente necesario.

```
1 struct SegTreeLazy {
2     int n; vector<ll> tree, lazy;
3     SegTreeLazy(int n) : n(n), tree(4 * n, 0), lazy(4 * n,
4         0) {}
5     void push(int node, int start, int end) {
6         if (lazy[node] != 0) {
7             tree[node] += (end - start + 1) * lazy[node];
8             if (start != end) {
9                 lazy[2 * node] += lazy[node];
10                lazy[2 * node + 1] += lazy[node];
11            }
12            lazy[node] = 0;
13        }
14    }
15    void updateRange(int node, int start, int end, int l,
16        int r, ll val) {
17        push(node, start, end);
18        if (start > end || start > r || end < l) return;
19        if (start >= l && end <= r) {
20            lazy[node] += val;
21            push(node, start, end);
22            return;
23        }
24        int mid = (start + end) / 2;
25        updateRange(2 * node, start, mid, l, r, val);
26        updateRange(2 * node + 1, mid + 1, end, l, r, val);
27        tree[node] = tree[2 * node] + tree[2 * node + 1];
28    }
29    ll queryRange(int node, int start, int end, int l, int
30        r) {
31        push(node, start, end);
32        if (r < start || end < l) return 0;
33        if (l <= start && end <= r) return tree[node];
34        int mid = (start + end) / 2;
35        return queryRange(2 * node, start, mid, l, r) +
36            queryRange(2 * node + 1, mid + 1, end, l, r);
37    }
38 }
```

```

29     if (start > end || start > r || end < 1) return 0;
30     if (start >= 1 && end <= r) return tree[node];
31     int mid = (start + end) / 2;
32     return queryRange(2 * node, start, mid, 1, r) +
33         queryRange(2 * node + 1, mid + 1, end, 1, r);
34 }
35 };

```

2.5 Merge Sort Tree

Complejidad de Tiempo:

- **Construcción:** $\mathcal{O}(N \log N)$
- **Consulta (Query):** $\mathcal{O}(\log^2 N)$ (utilizando búsqueda binaria interna)

Complejidad de Espacio: $\mathcal{O}(N \log N)$ (cada elemento del arreglo pertenece a $\log N$ vectores intermedios).

Estructura donde cada nodo del Segment Tree almacena un vector ordenado con todos los elementos de su subrango. Útil para consultas del tipo “cuántos números menores a K existen en el rango $[L, R]$ ”.

```

1 struct MergeSortTree {
2     int n; vector<vector<int>> tree;
3     MergeSortTree(int n) : n(n), tree(4 * n) {}
4     void build(const vector<int>& arr, int node, int start
5         , int end) {
6         if (start == end) {
7             tree[node] = {arr[start]};
8             return;
9         }
10        int mid = (start + end) / 2;
11        build(arr, 2 * node, start, mid);
12        build(arr, 2 * node + 1, mid + 1, end);
13
14        // Combina los dos subarreglos ordenados en  $\mathcal{O}(R-L+1)$ 
15        merge(tree[2 * node].begin(), tree[2 * node].end(),
16            tree[2 * node + 1].begin(), tree[2 * node + 1].end(),
17            back_inserter(tree[node]));
18
19        // Devuelve la cantidad de elementos estrictamente
20        // menores que K en  $[1, r]$ 
21        int query(int node, int start, int end, int l, int r,
22            int k) {
23            if (r < start || end < 1) return 0;
24            if (l <= start && end <= r) {
25                return lower_bound(tree[node].begin(), tree[
26                    node].end(), k) - tree[node].begin();
27            }
28            int mid = (start + end) / 2;
29            return query(2 * node, start, mid, l, r, k) +
30                query(2 * node + 1, mid + 1, end, l, r, k);
31        }
32    };
33 };

```

2.6 Sparse Table (RMQ Estático)

Complejidad de Tiempo:

- **Construcción:** $\mathcal{O}(N \log N)$
- **Consulta (Query):** $\mathcal{O}(1)$

Complejidad de Espacio: $\mathcal{O}(N \log N)$

Ideal para consultas de rango sobre funciones *idempotentes* (min, max, gcd, AND, OR) cuando el arreglo **no cambia**. Permite responder en $\mathcal{O}(1)$ usando dos ventanas de potencia de dos que se solapan.

```

1 struct SparseTable {
2     vector<vector<int>> table;
3     vector<int> logs;
4     int n;
5     SparseTable(const vector<int>& arr) {
6         n = arr.size();
7         logs.resize(n + 1);
8         for (int i = 2; i <= n; i++) logs[i] = logs[i / 2]
9             + 1;
10        int K = logs[n] + 1;
11        table.assign(K, vector<int>(n));
12        table[0] = arr;
13        for (int j = 1; j < K; j++)
14            for (int i = 0; i + (1 << j) <= n; i++)
15                table[j][i] = min(table[j-1][i], table[j-1][i + (1 << (j-1))]);
16    }
17    // Mínimo en el rango [l, r] (0-indexado, inclusivo)
18    int query(int l, int r) {
19        int j = logs[r - l + 1];
20        return min(table[j][l], table[j][r - (1 << j) + 1]);
21    }
22 };

```

2.7 DSU con Unión por Tamaño (Rollback-Ready)

Complejidad de Tiempo: $\mathcal{O}(\log N)$ por operación (sin compresión de caminos, para permitir deshacer).

Complejidad de Espacio: $\mathcal{O}(N)$

Variante de DSU que evita la compresión de caminos para poder deshacer uniones (*rollback*), útil en algoritmos offline como Kruskal reconstruible o consultas persistentes de conectividad.

```

1 struct DSURollback {
2     vector<int> parent, sz;
3     vector<pair<int,int>> history; // (nodo modificado,
4         // valor previo del parent)
5     DSURollback(int n) { parent.resize(n); sz.assign(n, 1);
6         ; iota(parent.begin(), parent.end(), 0); }
7     int find(int i) { while (parent[i] != i) i = parent[i];
8         return i; }
9     bool unite(int i, int j) {
10        int ri = find(i), rj = find(j);
11        if (ri == rj) { history.push_back({-1, -1});
12            return false; }
13        if (sz[ri] < sz[rj]) swap(ri, rj);
14        history.push_back({rj, parent[rj]});
15        parent[rj] = ri; sz[ri] += sz[rj];
16        return true;
17    }
18    void rollback() {
19        auto [node, prevParent] = history.back(); history.
20        pop_back();
21        if (node == -1) return;
22        sz[parent[node]] -= sz[node];
23        parent[node] = prevParent;
24    }
25 };

```

2.8 Trie (Árbol de Prefijos)

Complejidad de Tiempo: $\mathcal{O}(L)$ por inserción/consulta, donde L es la longitud de la cadena o el número de bits.

Complejidad de Espacio: $\mathcal{O}(N \cdot L)$

Estructura para almacenar conjuntos de cadenas o números binarios y responder consultas de prefijos comunes. La variante mostrada trabaja sobre bits de enteros de 30 posiciones, útil para problemas de XOR máximo.

```

1 struct TrieXor {

```

```
2 struct Node { int child[2] = {-1, -1}; };
3 vector<Node> trie;
4 static const int BITS = 30;
5 TrieXor() { trie.push_back(Node()); }
6 void insert(int num) {
7     int cur = 0;
8     for (int i = BITS; i >= 0; i--) {
9         int b = (num >> i) & 1;
10        if (trie[cur].child[b] == -1) {
11            trie[cur].child[b] = trie.size();
12            trie.push_back(Node());
13        }
14        cur = trie[cur].child[b];
15    }
16 }
17 // Devuelve el maximo XOR posible entre 'num' y algun
18 // valor insertado
19 int maxXor(int num) {
20     int cur = 0, res = 0;
21     for (int i = BITS; i >= 0; i--) {
22         int b = (num >> i) & 1;
23         int want = 1 - b;
24         if (trie[cur].child[want] != -1) {
25             res |= (1 << i);
26             cur = trie[cur].child[want];
27         } else {
28             cur = trie[cur].child[b];
29         }
30     }
31     return res;
32 };
```

3. Lower Bound y Upper Bound

3.1 Conceptos y Uso Básico en STL

Complejidad de Tiempo: $\mathcal{O}(\log N)$ sobre contenedores de acceso aleatorio (vector, array).

Requisito: el rango debe estar **ordenado** respecto al criterio usado.

`lower_bound(first, last, val)` devuelve un iterador al **primer** elemento $\geq val$.

`upper_bound(first, last, val)` devuelve un iterador al **primer** elemento $> val$.

Ambos devuelven `last` si no existe tal elemento. La diferencia entre ambos resultados da la cantidad de ocurrencias exactas de `val`.

```
1 vector<int> v = {1, 3, 3, 3, 5, 7, 9};
2 auto lb = lower_bound(v.begin(), v.end(), 3); // apunta al
  primer 3 (índice 1)
3 auto ub = upper_bound(v.begin(), v.end(), 3); // apunta al
  primer >3 (índice 4, valor 5)
4 int idx_lb = lb - v.begin(); // 1
5 int idx_ub = ub - v.begin(); // 4
6 int cantidad_de_3 = ub - lb; // 3
7 bool existe = (lb != v.end() && *lb == 3); // forma
  estandar de checar existencia
```

Patrones Adaptativos Rápidos

Snippets listos para pegar y adaptar según lo que se necesite consultar:

```
1 // Contar cuantos elementos son <= x
2 int cnt_le(vector<int>& v, int x) { return upper_bound(v.
  begin(), v.end(), x) - v.begin(); }
3 // Contar cuantos elementos son < x
4 int cnt_lt(vector<int>& v, int x) { return lower_bound(v.
  begin(), v.end(), x) - v.begin(); }
5 // Contar cuantos elementos son >= x
6 int cnt_ge(vector<int>& v, int x) { return v.end() -
  lower_bound(v.begin(), v.end(), x); }
7 // Contar cuantos elementos son > x
8 int cnt_gt(vector<int>& v, int x) { return v.end() -
  upper_bound(v.begin(), v.end(), x); }
9 // Contar cuantos elementos hay en el rango [lo, hi]
10 int cnt_range(vector<int>& v, int lo, int hi) {
11   return upper_bound(v.begin(), v.end(), hi) -
    lower_bound(v.begin(), v.end(), lo);
12 }
13 // Mayor elemento <= x (floor). Retorna -1 si no existe
14 int floor_val(vector<int>& v, int x) {
15   auto it = upper_bound(v.begin(), v.end(), x);
16   if (it == v.begin()) return -1;
17   return *prev(it);
18 }
19 // Menor elemento >= x (ceil). Retorna -1 si no existe
20 int ceil_val(vector<int>& v, int x) {
21   auto it = lower_bound(v.begin(), v.end(), x);
22   if (it == v.end()) return -1;
23   return *it;
24 }
```

3.2 Búsqueda con Comparador Personalizado

Complejidad de Tiempo: $\mathcal{O}(\log N)$

Cuando el contenedor guarda `pair`, `struct` u otro tipo compuesto, se busca por un campo específico pasando un comparador (lambda) como cuarto argumento. **Importante:** el comparador debe seguir el mismo orden estricto usado al ordenar el contenedor.

```
1 struct Persona { int edad; string nombre; };
2 vector<Persona> gente = { {18,"Ana"}, {20,"Luis"}, {20,"
  Eva"}, {25,"Tom"} };
3 // gente esta ordenado por 'edad'
4 // Buscar primera persona con edad >= 20 (comparando 'val'
  contra el campo del struct)
5 auto it = lower_bound(gente.begin(), gente.end(), 20,
6   [](const Persona& p, int val) { return p.edad < val;
  });
7 // Buscar primera persona con edad > 20
8 auto it2 = upper_bound(gente.begin(), gente.end(), 20,
9   [](int val, const Persona& p) { return val < p.edad;
  });
10 // Sirve igual con vector<pair<int,int>> ordenado por '
  first'
11 vector<pair<int,int>> vp = {{1,5},{3,8},{3,2},{7,1}};
12 auto it3 = lower_bound(vp.begin(), vp.end(), make_pair(3,
  INT_MIN));
```

3.3 lower_bound / upper_bound en set y multiset

Complejidad de Tiempo: $\mathcal{O}(\log N)$ usando el método miembro (**NO** usar la función libre `std::lower_bound` sobre estos contenedores: al no tener iteradores de acceso aleatorio, degrada a $\mathcal{O}(N)$).

`set/multiset` ya están ordenados internamente; siempre se debe llamar al **método miembro** `.lower_bound()` / `.upper_bound()` para mantener eficiencia.

```
1 multiset<int> ms = {1, 3, 3, 5, 7};
2 auto it = ms.lower_bound(3); // 0(log N), correcto
3 // auto it_malo = lower_bound(ms.begin(), ms.end(), 3); //
  O(N), EVITAR
4 // Predecesor y sucesor de x (aunque x no este en el set)
5 int predecesor(multiset<int>& s, int x) {
6   auto it = s.lower_bound(x);
7   if (it == s.begin()) return INT_MIN; // no hay
    predecesor
8   return *prev(it);
9 }
10 int sucesor(multiset<int>& s, int x) {
11   auto it = s.upper_bound(x);
12   if (it == s.end()) return INT_MAX; // no hay sucesor
13   return *it;
14 }
15 // Eliminar SOLO una ocurrencia de x en un multiset (erase
  (x) borra TODAS)
16 auto itDel = ms.find(3);
17 if (itDel != ms.end()) ms.erase(itDel);
```

3.4 Búsqueda Binaria Genérica sobre Predicado (Adaptativa)

Complejidad de Tiempo: $\mathcal{O}(\log N \cdot f)$, donde f es el costo de evaluar el predicado (*check*).

Complejidad de Espacio: $\mathcal{O}(1)$

Esta plantilla es la más **reutilizable** en competencia: sirve para cualquier situación monótona (no solo arreglos ordenados), como *binary search on answer*. Solo se debe definir la función `check(x)` que sea `true` para todo $x \geq$ respuesta y `false` antes de ella (patrón FFFF...TTTT).

```
1 // Encuentra el menor x en [lo, hi] tal que check(x) ==
  true
2 // (asume que check es monotonica: false,false,...,false,
  true,true,...,true)
3 template <typename T, typename F>
4 T binary_search_first_true(T lo, T hi, F check) {
```

```

5     T ans = hi + 1; // valor centinela si nunca se cumple
6     while (lo <= hi) {
7         T mid = lo + (hi - lo) / 2;
8         if (check(mid)) { ans = mid; hi = mid - 1; }
9         else lo = mid + 1;
10    }
11    return ans;
12 }
13 // Ejemplo de uso: menor capacidad de barco para repartir
14 // paquetes en D dias
15 // bool check(int capacidad) { ... simula si alcanza con
16 // esa capacidad ... }
17 // int resp = binary_search_first_true(minCap, maxCap,
18 // check);
19 // Variante para punto flotante (numero fijo de
20 // iteraciones en vez de lo<=hi)
21 template <typename F>
22 double binary_search_real(double lo, double hi, F check,
23 int iters = 100) {
24     for (int i = 0; i < iters; i++) {
25         double mid = (lo + hi) / 2;
26         if (check(mid)) hi = mid; else lo = mid;
27     }
28     return lo;
29 }

```

3.5 Implementación Manual (Referencia Interna)

Complejidad de Tiempo: $\mathcal{O}(\log N)$

Útil para entender qué hace la STL por dentro, o para adaptarla a estructuras sin iteradores estándar (ej. arreglos con índices *custom*, matrices implícitas, respuestas indexadas por función).

```

1 // Version manual de lower_bound: primer indice con arr[i]
2 // >= target
3 int my_lower_bound(const vector<int>& arr, int target) {
4     int lo = 0, hi = (int)arr.size(); // hi = size (no
5     size-1): rango [lo, hi)
6     while (lo < hi) {
7         int mid = lo + (hi - lo) / 2;
8         if (arr[mid] < target) lo = mid + 1;
9         else hi = mid;
10    }
11    return lo; // si lo == arr.size(), no existe tal
12    elemento
13 }
14 // Version manual de upper_bound: primer indice con arr[i]
15 // > target
16 int my_upper_bound(const vector<int>& arr, int target) {
17     int lo = 0, hi = (int)arr.size();
18     while (lo < hi) {
19         int mid = lo + (hi - lo) / 2;
20         if (arr[mid] <= target) lo = mid + 1;
21         else hi = mid;
22     }
23    return lo;
24 }

```

4. Algoritmos de Grafos

4.1 Tabla de Decisión Rápida

Antes de programar, ubica el problema en esta tabla. Ahorra minutos valiosos en el momento del concurso.

Pista en el enunciado	Algoritmo
Menor cantidad de pasos/aristas, sin pesos	BFS
Pesos todos iguales a 1 o dos valores fijos	BFS 0/1 (deque)
Varios orígenes simultáneos	BFS multi-fuente
Camino mínimo con pesos ≥ 0	Dijkstra
Camino mínimo con pesos negativos	Bellman-Ford
Todos los pares de distancias, $V \leq 500$	Floyd-Warshall
¿Se puede llegar de A a B?	DFS o BFS
Contar componentes conexas / islas en grid	DFS o BFS desde cada no visitado
¿Es bipartito? / 2 colores	BFS/DFS coloreando
Orden de tareas con dependencias	Topological Sort (Kahn o DFS)
Detectar ciclo	DFS (colores) o Kahn
Árbol de conexión de costo mínimo	Kruskal o Prim
Componentes fuertemente conexas (dirigido)	Tarjan / Kosaraju
Nodo/arista crítica que desconecta el grafo	Puentes y Puntos de Articulación
Ancastro común de dos nodos en árbol	LCA (Binary Lifting)
Satisfacibilidad de restricciones OR con 2 opciones	2-SAT
Flujo máximo / corte mínimo	Edmonds-Karp
Grid/matriz con movimientos (arriba/abajo/izq/der)	Grafo implícito: BFS/DFS sobre celdas

4.2 Plantilla Universal de Grafo

Copiar y pegar según lo que se necesite. Usar SIEMPRE la misma variable adj en toda la solución para no confundirse a mitad de la competencia.

```
1 int n, m; // nodos y aristas
2 vector<vector<int>>> adj; // grafo NO ponderado
3 vector<vector<pair<int,int>>>> adjW; // grafo ponderado: {vecino, peso}
4
5 void leerGrafoNoDirigido(bool ponderado) {
6     cin >> n >> m;
7     adj.assign(n + 1, {});
8     adjW.assign(n + 1, {});
9     for (int i = 0; i < m; i++) {
10         int u, v; cin >> u >> v;
11         if (ponderado) {
12             int w; cin >> w;
13             adjW[u].push_back({v, w});
14             adjW[v].push_back({u, w});
15         } else {
16             adj[u].push_back(v);
17             adj[v].push_back(u); // quitar esta linea si es dirigido
18         }
19     }
20 }
21 // IMPORTANTE (multi-test): siempre limpiar/reasignar adj y adjW entre casos de prueba.
```

4.3 BFS (Búsqueda en Anchura)

Complejidad: $\mathcal{O}(V + E)$ tiempo, $\mathcal{O}(V)$ espacio.

Garantiza la distancia mínima en aristas cuando todos los pesos son 1 (o no hay pesos).

```
1 vector<int> dist_, parent_;
2 void bfs(int start, int n, vector<vector<int>>& adj) {
3     dist_.assign(n + 1, -1);
4     parent_.assign(n + 1, -1);
5     queue<int> q;
6     dist_[start] = 0;
7     q.push(start);
8     while (!q.empty()) {
9         int u = q.front(); q.pop();
10        for (int v : adj[u]) {
11            if (dist_[v] == -1) {
12                dist_[v] = dist_[u] + 1;
13                parent_[v] = u;
14                q.push(v);
15            }
16        }
17    }
18 }
19 // Reconstruir camino start -> target (vacío si no hay camino)
20 vector<int> reconstruirCamino(int target) {
21     if (dist_[target] == -1) return {};
22     vector<int> camino;
23     for (int v = target; v != -1; v = parent_[v]) camino.push_back(v);
24     reverse(camino.begin(), camino.end());
25     return camino;
26 }
```

4.3.1 BFS Multi-Fuente

Útil cuando hay varios orígenes que "expanden" simultáneamente (ej. varias fuentes de fuego, virus, etc.). Basta con meter todos los orígenes a la cola desde el inicio con distancia 0.

```
1 vector<int> bfsMultiFuente(int n, vector<vector<int>>& adj, vector<int>& fuentes) {
2     vector<int> dist(n + 1, -1);
3     queue<int> q;
4     for (int s : fuentes) { dist[s] = 0; q.push(s); }
5     while (!q.empty()) {
6         int u = q.front(); q.pop();
7         for (int v : adj[u]) {
8             if (dist[v] == -1) { dist[v] = dist[u] + 1; q.push(v); }
9         }
10    }
11    return dist;
12 }
```

4.3.2 BFS 0/1 (Deque)

Complejidad: $\mathcal{O}(V + E)$

Cuando las aristas solo tienen peso 0 o 1 (o dos valores fijos), es más rápido que Dijkstra y evita la cola de prioridad. Aristas de peso 0 se insertan al frente del deque, las de peso 1 al final.

```
1 vector<int> bfs01(int start, int n, vector<vector<pair<int,int>>>& adjW) {
2     vector<int> dist(n + 1, INT_MAX);
3     deque<int> dq;
4     dist[start] = 0;
5     dq.push_back(start);
6     while (!dq.empty()) {
7         int u = dq.front(); dq.pop_front();
8         for (auto [v, w] : adjW[u]) { // w debe ser 0 o 1
9             if (dist[u] + w < dist[v]) {
```

```

10         dist[v] = dist[u] + w;
11         if (w == 0) dq.push_front(v);
12         else dq.push_back(v);
13     }
14 }
15 }
16 return dist;
17 }

```

4.4 DFS (Búsqueda en Profundidad)

Complejidad: $\mathcal{O}(V + E)$ tiempo, $\mathcal{O}(V)$ espacio.

Cuidado: con $n \gtrsim 10^5$ y grafos tipo "cadena", la versión recursiva puede tronar por stack overflow. Usar la versión iterativa en ese caso.

```

1 vector<bool> visitado;
2 void dfs(int u, vector<vector<int>>& adj) {
3     visitado[u] = true;
4     // procesar u aqui
5     for (int v : adj[u]) {
6         if (!visitado[v]) dfs(v, adj);
7     }
8 }

```

4.4.1 DFS Iterativo (evita stack overflow)

```

1 void dfsIterativo(int start, int n, vector<vector<int>>&
    adj) {
2     vector<bool> visitado(n + 1, false);
3     stack<int> st;
4     st.push(start);
5     while (!st.empty()) {
6         int u = st.top(); st.pop();
7         if (visitado[u]) continue;
8         visitado[u] = true;
9         // procesar u aqui
10        for (int v : adj[u]) {
11            if (!visitado[v]) st.push(v);
12        }
13    }
14 }

```

4.5 Componentes Conexas y Grafos Bipartitos

Complejidad: $\mathcal{O}(V + E)$

```

1 // Cuenta componentes conexas y asigna a cada nodo su id
  // de componente
2 int contarComponentes(int n, vector<vector<int>>& adj,
    vector<int>& compId) {
3     compId.assign(n + 1, -1);
4     int comp = 0;
5     for (int i = 1; i <= n; i++) {
6         if (compId[i] != -1) continue;
7         queue<int> q; q.push(i); compId[i] = comp;
8         while (!q.empty()) {
9             int u = q.front(); q.pop();
10            for (int v : adj[u]) {
11                if (compId[v] == -1) { compId[v] = comp; q
                .push(v); }
12            }
13        }
14        comp++;
15    }
16    return comp;
17 }
18 // Determina si el grafo es bipartito (coloreable con 2
  // colores)
19 bool esBipartito(int n, vector<vector<int>>& adj) {
20     vector<int> color(n + 1, -1);

```

```

21     for (int i = 1; i <= n; i++) {
22         if (color[i] != -1) continue;
23         queue<int> q; q.push(i); color[i] = 0;
24         while (!q.empty()) {
25             int u = q.front(); q.pop();
26             for (int v : adj[u]) {
27                 if (color[v] == -1) { color[v] = 1 - color
                [u]; q.push(v); }
28                 else if (color[v] == color[u]) return
                false;
29             }
30         }
31     }
32     return true;
33 }

```

4.6 Detección de Ciclos

Complejidad: $\mathcal{O}(V + E)$

4.6.1 Grafo NO Dirigido

Se compara con el padre para no contar la misma arista dos veces.

```

1 vector<bool> visitado;
2 bool cicloNoDirigido(int u, int padre, vector<vector<int>
    >>& adj) {
3     visitado[u] = true;
4     for (int v : adj[u]) {
5         if (!visitado[v]) {
6             if (cicloNoDirigido(v, u, adj)) return true;
7         } else if (v != padre) {
8             return true;
9         }
10    }
11    return false;
12 }
13 bool tieneCicloNoDirigido(int n, vector<vector<int>>& adj)
    {
14     visitado.assign(n + 1, false);
15     for (int i = 1; i <= n; i++)
16         if (!visitado[i] && cicloNoDirigido(i, -1, adj))
17             return true;
18     return false;
19 }

```

4.6.2 Grafo Dirigido (3 colores)

Un ciclo existe solo si se llega a un nodo que sigue "en la pila de recursión" (gris), no uno ya terminado (negro).

```

1 // 0 = blanco, 1 = gris (en el camino actual), 2 = negro (
  // terminado)
2 vector<int> color_;
3 bool cicloDirigido(int u, vector<vector<int>>& adj) {
4     color_[u] = 1;
5     for (int v : adj[u]) {
6         if (color_[v] == 1) return true;
7         if (color_[v] == 0 && cicloDirigido(v, adj))
8             return true;
9     }
10    color_[u] = 2;
11    return false;
12 }
13 bool tieneCicloDirigido(int n, vector<vector<int>>& adj) {
14     color_.assign(n + 1, 0);
15     for (int i = 1; i <= n; i++)
16         if (color_[i] == 0 && cicloDirigido(i, adj))
17             return true;
18     return false;
19 }

```

4.7 Ordenamiento Topológico

Solo aplica a DAGs (grafos dirigidos acíclicos).

4.7.1 Kahn (BFS) — Recomendado

Complejidad: $\mathcal{O}(V + E)$

Ventaja: detecta ciclos automáticamente (si `orden.size() != n`, no es DAG).

```
1 vector<int> topoSortKahn(int n, vector<vector<int>>& adj)
2 {
3     vector<int> indeg(n + 1, 0);
4     for (int u = 1; u <= n; u++) for (int v : adj[u])
5         indeg[v]++;
6     queue<int> q;
7     for (int i = 1; i <= n; i++) if (indeg[i] == 0) q.push(i);
8     vector<int> orden;
9     while (!q.empty()) {
10         int u = q.front(); q.pop();
11         orden.push_back(u);
12         for (int v : adj[u]) if (--indeg[v] == 0) q.push(v);
13     }
14     if ((int)orden.size() != n) return {}; // hay ciclo, no es DAG
15     return orden;
16 }
```

4.7.2 DFS (post-orden invertido)

```
1 vector<bool> visitadoTopo;
2 vector<int> ordenTopo;
3 void dfsTopo(int u, vector<vector<int>>& adj) {
4     visitadoTopo[u] = true;
5     for (int v : adj[u]) if (!visitadoTopo[v]) dfsTopo(v, adj);
6     ordenTopo.push_back(u);
7 }
8 vector<int> topoSortDFS(int n, vector<vector<int>>& adj) {
9     visitadoTopo.assign(n + 1, false);
10    ordenTopo.clear();
11    for (int i = 1; i <= n; i++) if (!visitadoTopo[i]) dfsTopo(i, adj);
12    reverse(ordenTopo.begin(), ordenTopo.end());
13    return ordenTopo; // NO valida ciclos, verificar con tieneCicloDirigido primero
14 }
```

4.8 Caminos Mínimos con Pesos

4.8.1 Dijkstra (pesos no negativos)

Complejidad: $\mathcal{O}((V + E) \log V)$

```
1 const ll INF_LL = 1e18;
2 vector<ll> dijkstra(int src, int n, vector<vector<pair<int, ll>>& adj) {
3     vector<ll> dist(n + 1, INF_LL);
4     priority_queue<pair<ll, int>, vector<pair<ll, int>>, greater<>> pq;
5     dist[src] = 0; pq.push({0, src});
6     while (!pq.empty()) {
7         auto [d, u] = pq.top(); pq.pop();
8         if (d > dist[u]) continue;
9         for (auto [v, w] : adj[u]) {
10             if (dist[u] + w < dist[v]) {
11                 dist[v] = dist[u] + w;
12                 pq.push({dist[v], v});
13             }
14         }
15     }
16     return dist;
17 }
```

4.8.2 Bellman-Ford (permite pesos negativos)

Complejidad: $\mathcal{O}(V \cdot E)$

```
1 struct Edge { int u, v; ll w; };
2 pair<vector<ll>, bool> bellmanFord(int src, int n, vector<
3     Edge>& edges) {
4     vector<ll> dist(n + 1, INF_LL);
5     dist[src] = 0;
6     for (int i = 0; i < n - 1; i++)
7         for (auto& e : edges)
8             if (dist[e.u] != INF_LL && dist[e.u] + e.w < dist[e.v])
9                 dist[e.v] = dist[e.u] + e.w;
10    bool cicloNegativo = false;
11    for (auto& e : edges)
12        if (dist[e.u] != INF_LL && dist[e.u] + e.w < dist[e.v]) {
13            cicloNegativo = true; break; }
14    return {dist, cicloNegativo};
15 }
```

4.8.3 Floyd-Warshall (todos los pares)

Complejidad: $\mathcal{O}(V^3)$. Práctico para $V \lesssim 500$.

```
1 void floydWarshall(vector<vector<ll>>& dist, int n) {
2     // dist[i][j] = peso i->j, INF si no hay arista, 0 si i == j
3     for (int k = 0; k < n; k++)
4         for (int i = 0; i < n; i++)
5             for (int j = 0; j < n; j++)
6                 if (dist[i][k] < INF_LL && dist[k][j] < INF_LL)
7                     dist[i][j] = min(dist[i][j], dist[i][k] + dist[k][j]);
8     // Ciclo negativo si dist[i][i] < 0 para algun i
9 }
```

4.9 Árbol de Expansión Mínima (MST)

4.9.1 Kruskal (recomendado, más fácil de codear)

Complejidad: $\mathcal{O}(E \log E)$. Requiere DSU (ver capítulo de Estructuras de Datos).

```
1 struct EdgeW { int u, v; ll w; bool operator<(const EdgeW& o) const { return w < o.w; } };
2 ll kruskal(int n, vector<EdgeW>& edges) {
3     sort(edges.begin(), edges.end());
4     DSU dsu(n + 1);
5     ll costoTotal = 0; int aristasUsadas = 0;
6     for (auto& e : edges) {
7         if (dsu.find(e.u) != dsu.find(e.v)) {
8             dsu.unite(e.u, e.v);
9             costoTotal += e.w;
10            aristasUsadas++;
11        }
12    }
13    // Si aristasUsadas != n - 1, el grafo no es conexo
14    return costoTotal;
15 }
```

4.9.2 Prim (mejor para grafos densos)

Complejidad: $\mathcal{O}(E \log V)$

```
1 ll prim(int n, vector<vector<pair<int, ll>>& adj) {
2     vector<bool> inMST(n + 1, false);
3     priority_queue<pair<ll, int>, vector<pair<ll, int>>, greater<>> pq;
4     pq.push({0, 1}); // asumiendo raiz = nodo 1
5     ll costoTotal = 0;
6     while (!pq.empty()) {
7         auto [w, u] = pq.top(); pq.pop();
8         if (inMST[u]) continue;
9         inMST[u] = true; costoTotal += w;
10        for (auto [v, peso] : adj[u]) if (!inMST[v]) pq.push({peso, v});
11    }
12 }
```

```

11 }
12 return costoTotal;
13 }

```

4.10 Componentes Fuertemente Conexas: Tarjan

Complejidad: $\mathcal{O}(V + E)$. Encuentra SCC de un grafo dirigido en una sola pasada DFS.

```

1 int timerScc; vector<vector<int>>> adjScc;
2 vector<int> disc, low, comp;
3 vector<bool> onStack; stack<int> stScc; int sccCount;
4 void tarjanDfs(int u) {
5     disc[u] = low[u] = timerScc++;
6     stScc.push(u); onStack[u] = true;
7     for (int v : adjScc[u]) {
8         if (disc[v] == -1) { tarjanDfs(v); low[u] = min(
9             low[u], low[v]);
10            else if (onStack[v]) low[u] = min(low[u], disc[v]);
11        }
12        if (low[u] == disc[u]) {
13            while (true) {
14                int v = stScc.top(); stScc.pop(); onStack[v] =
15                    false;
16                comp[v] = sccCount;
17                if (v == u) break;
18            }
19            sccCount++;
20        }
21    }
22 void tarjanSCC(int n) {
23     disc.assign(n + 1, -1); low.assign(n + 1, -1);
24     comp.assign(n + 1, -1); onStack.assign(n + 1, false);
25     timerScc = 0; sccCount = 0;
26     for (int i = 1; i <= n; i++) if (disc[i] == -1)
27         tarjanDfs(i);
28 }

```

4.11 Puentes y Puntos de Articulación

Complejidad: $\mathcal{O}(V + E)$. Aristas/vértices cuya eliminación desconecta el grafo (grafo NO dirigido).

```

1 int timerBridge; vector<vector<int>>> adjBridge;
2 vector<int> discB, lowB, parentB;
3 vector<bool> esArticulacion;
4 vector<pair<int, int>> puentes;
5 void bridgeDfs(int u) {
6     discB[u] = lowB[u] = timerBridge++;
7     int hijos = 0;
8     for (int v : adjBridge[u]) {
9         if (v == parentB[u]) continue;
10        if (discB[v] != -1) {
11            lowB[u] = min(lowB[u], discB[v]);
12        } else {
13            hijos++;
14            parentB[v] = u;
15            bridgeDfs(v);
16            lowB[u] = min(lowB[u], lowB[v]);
17            if (lowB[v] > discB[u]) puentes.push_back({u,
18                v});
19            if (parentB[u] != -1 && lowB[v] >= discB[u])
20                esArticulacion[u] = true;
21        }
22    }
23    if (parentB[u] == -1 && hijos > 1) esArticulacion[u] =
24        true; // caso raiz
25 }
26 void encontrarPuentesYArticulacion(int n) {
27     discB.assign(n + 1, -1); lowB.assign(n + 1, -1);
28     parentB.assign(n + 1, -1); esArticulacion.assign(n +
29         1, false);
30     timerBridge = 0;

```

```

27 for (int i = 1; i <= n; i++) if (discB[i] == -1)
28     bridgeDfs(i);

```

4.12 Ancestro Común Más Bajo (LCA) — Binary Lifting

Complejidad: $\mathcal{O}(V \log V)$ preprocesamiento, $\mathcal{O}(\log V)$ por consulta.

```

1 int LOG_;
2 vector<vector<int>>> upLca, adjLca;
3 vector<int> depthLca;
4 void lcaDfs(int u, int p) {
5     upLca[u][0] = p;
6     for (int j = 1; j < LOG_; j++)
7         upLca[u][j] = (upLca[u][j-1] == -1) ? -1 : upLca[
8             upLca[u][j-1]][j-1];
9     for (int v : adjLca[u]) if (v != p) { depthLca[v] =
10         depthLca[u] + 1; lcaDfs(v, u); }
11 }
12 void lcaInit(int n, int raiz) {
13     LOG_ = ceil(log2(max(2, n))) + 1;
14     upLca.assign(n + 1, vector<int>(LOG_, -1));
15     depthLca.assign(n + 1, 0);
16     lcaDfs(raiz, -1);
17 }
18 int lcaQuery(int u, int v) {
19     if (depthLca[u] < depthLca[v]) swap(u, v);
20     int diff = depthLca[u] - depthLca[v];
21     for (int j = 0; j < LOG_; j++) if ((diff >> j) & 1) u
22         = upLca[u][j];
23     if (u == v) return u;
24     for (int j = LOG_ - 1; j >= 0; j--)
25         if (upLca[u][j] != upLca[v][j]) { u = upLca[u][j];
26             v = upLca[v][j]; }
27     return upLca[u][0];
28 }
29 // Distancia entre u y v en el arbol:
30 // int distArbol(int u, int v) { int l = lcaQuery(u, v);
31     return depthLca[u] + depthLca[v] - 2 * depthLca[l]; }

```

4.13 2-SAT

Complejidad: $\mathcal{O}(V + E)$

```

1 struct TwoSAT {
2     int n;
3     vector<vector<int>>> adj, radj;
4     vector<int> comp, order_, assign_;
5     vector<bool> visited;
6     TwoSAT(int n) : n(n), adj(2*n), radj(2*n), comp(2*n,
7         -1), visited(2*n, false) {}
8     int idx(int var, bool val) { return 2*var + (val ? 1 :
9         0); }
10    void addClause(int var1, bool val1, int var2, bool
11        val2) {
12        adj[idx(var1, !val1)].push_back(idx(var2, val2));
13        adj[idx(var2, !val2)].push_back(idx(var1, val1));
14        radj[idx(var2, val2)].push_back(idx(var1, !val1));
15        radj[idx(var1, val1)].push_back(idx(var2, !val2));
16    }
17    void dfs1(int u) {
18        visited[u] = true;
19        for (int v : adj[u]) if (!visited[v]) dfs1(v);
20    }
21    void dfs2(int u, int c) {
22        comp[u] = c;
23        for (int v : radj[u]) if (comp[v] == -1) dfs2(v, c
24            );
25    }
26    bool solve() {
27        for (int i = 0; i < 2*n; i++) if (!visited[i])
28            dfs1(i);
29        int c = 0;
30        for (int i = 2*n - 1; i >= 0; i--) {
31            int u = order_[i];
32            if (comp[u] == -1) dfs2(u, c++);

```

```

29     }
30     assign_.assign(n, -1);
31     for (int i = 0; i < n; i++) {
32         if (comp[idx(i,true)] == comp[idx(i,false)])
33             return false;
34         assign_[i] = comp[idx(i,true)] > comp[idx(i,
35             false)];
36     }
37     return true;
38 };

```

4.14 Flujo Máximo: Edmonds-Karp

Complejidad: $\mathcal{O}(VE^2)$

```

1  const int INF = 1e9;
2  int nFlujo; vector<vector<int>> capacidad, adjFlujo;
3  int bfsFlujo(int s, int t, vector<int>& parent) {
4      fill(parent.begin(), parent.end(), -1); parent[s] =
5      -2;
6      queue<pair<int,int>> q; q.push({s, INF});
7      while (!q.empty()) {
8          auto [cur, flow] = q.front(); q.pop();
9          for (int next : adjFlujo[cur]) {
10             if (parent[next] == -1 && capacidad[cur][next]
11                 > 0) {
12                 parent[next] = cur;
13                 int newFlow = min(flow, capacidad[cur][
14                     next]);
15                 if (next == t) return newFlow;
16                 q.push({next, newFlow});
17             }
18         }
19     }
20     return 0;
21 }
22 int edmondsKarp(int s, int t) {
23     int flow = 0; vector<int> parent(nFlujo); int newFlow;
24     while ((newFlow = bfsFlujo(s, t, parent))) {
25         flow += newFlow; int cur = t;
26         while (cur != s) {
27             int prev = parent[cur];
28             capacidad[prev][cur] -= newFlow;
29             capacidad[cur][prev] += newFlow;
30             cur = prev;
31         }
32     }
33     return flow;
34 }

```

4.15 Grafos Implícitos en Grid/Matriz

Complejidad: $\mathcal{O}(R \cdot C)$ donde R, C son filas y columnas. Muy frecuente en competencia: laberintos, conteo de islas, agua/incendio expandiéndose. Cada celda es un nodo implícito, sin necesidad de construir adj explícitamente.

```

1  int R, C;
2  vector<string> grid; // grid[i][j] = caracter de la celda
3  vector<vector<int>> distGrid;
4  // 4 direcciones (arriba, abajo, izquierda, derecha)
5  int dx4[] = {-1, 1, 0, 0};
6  int dy4[] = {0, 0, -1, 1};
7  // 8 direcciones (agrega diagonales)
8  int dx8[] = {-1,-1,-1, 0, 0, 1, 1, 1};
9  int dy8[] = {-1, 0, 1,-1, 1,-1, 0, 1};
10
11 bool dentroDelGrid(int x, int y) { return x >= 0 && x < R
12     && y >= 0 && y < C; }
13
14 void bfsGrid(int startX, int startY, char obstaculo = '#')
15 {
16     distGrid.assign(R, vector<int>(C, -1));
17     queue<pair<int,int>> q;
18     distGrid[startX][startY] = 0;

```

```

17     q.push({startX, startY});
18     while (!q.empty()) {
19         auto [x, y] = q.front(); q.pop();
20         for (int d = 0; d < 4; d++) {
21             int nx = x + dx4[d], ny = y + dy4[d];
22             if (dentroDelGrid(nx, ny) && distGrid[nx][ny]
23                 == -1 && grid[nx][ny] != obstaculo) {
24                 distGrid[nx][ny] = distGrid[x][y] + 1;
25                 q.push({nx, ny});
26             }
27         }
28     }
29 }
30 // Contar componentes conexas en grid (ej: contar islas)
31 vector<vector<bool>> visitadoGrid;
32 void dfsGrid(int x, int y, char objetivo) {
33     if (!dentroDelGrid(x, y) || visitadoGrid[x][y] || grid
34         [x][y] != objetivo) return;
35     visitadoGrid[x][y] = true;
36     for (int d = 0; d < 4; d++) dfsGrid(x + dx4[d], y +
37         dy4[d], objetivo);
38 }
39 int contarIslas(char tierra = '1') {
40     visitadoGrid.assign(R, vector<bool>(C, false));
41     int islas = 0;
42     for (int i = 0; i < R; i++)
43         for (int j = 0; j < C; j++)
44             if (grid[i][j] == tierra && !visitadoGrid[i][j])
45                 { dfsGrid(i, j, tierra); islas++; }
46     return islas;
47 }

```

4.16 Errores Comunes que Cuestan Penalizaciones

- Olvidar resetear visitado/dist/adj entre casos de prueba (multi-test).
- No verificar límites de un grid/matriz antes de moverse a un vecino.
- Confundir grafo dirigido vs no dirigido al agregar aristas.
- Usar DFS recursivo en grafos gigantes ($n \gtrsim 10^5$) con forma de cadena $\rightarrow$ stack overflow. Usar BFS o DFS iterativo.
- En BFS: marcar visitado al **encolar**, no al desencolar (si no, el mismo nodo entra varias veces a la cola).
- Usar Dijkstra con pesos negativos (da resultados incorrectos): usar Bellman-Ford en ese caso.
- Olvidar que topoSortDFS no detecta ciclos por sí solo: verificar aciclicidad antes, o usar Kahn directamente.
- En 0/1 BFS, confundir el orden de push_front/push_back (0 al frente, 1 al final).

5. Cadenas (Strings)

5.1 KMP (Knuth-Morris-Pratt)

Complejidad de Tiempo: $\mathcal{O}(N+M)$, donde N es el texto y M el patrón.

Complejidad de Espacio: $\mathcal{O}(M)$

Busca todas las ocurrencias de un patrón dentro de un texto sin retroceder en el texto, usando una tabla de fallos (*prefix function*) precalculada sobre el patrón.

```
1 vector<int> build_prefix_function(const string& pat) {
2     int m = pat.size();
3     vector<int> pi(m, 0);
4     for (int i = 1; i < m; i++) {
5         int j = pi[i-1];
6         while (j > 0 && pat[i] != pat[j]) j = pi[j-1];
7         if (pat[i] == pat[j]) j++;
8         pi[i] = j;
9     }
10    return pi;
11 }
12 // Devuelve las posiciones (0-indexadas) en 'texto' donde
13 // inicia 'pat'
14 vector<int> kmp_search(const string& texto, const string&
15 pat) {
16     vector<int> pi = build_prefix_function(pat);
17     vector<int> ocurrencias;
18     int j = 0;
19     for (int i = 0; i < (int)texto.size(); i++) {
20         while (j > 0 && texto[i] != pat[j]) j = pi[j-1];
21         if (texto[i] == pat[j]) j++;
22         if (j == (int)pat.size()) {
23             ocurrencias.push_back(i - j + 1);
24             j = pi[j-1];
25         }
26     }
27     return ocurrencias;
28 }
```

5.2 Z-Function

Complejidad de Tiempo: $\mathcal{O}(N)$

Complejidad de Espacio: $\mathcal{O}(N)$

Para cada posición i , calcula la longitud del prefijo común más largo entre la cadena completa y el sufijo que empieza en i . Útil para *matching* de patrones (concatenando *pat* + *#* + *texto*) y detección de periodicidad.

```
1 vector<int> z_function(const string& s) {
2     int n = s.size();
3     vector<int> z(n, 0);
4     int l = 0, r = 0;
5     for (int i = 1; i < n; i++) {
6         if (i < r) z[i] = min(r - i, z[i - l]);
7         while (i + z[i] < n && s[z[i]] == s[i + z[i]]) z[i]++;
8         if (i + z[i] > r) { l = i; r = i + z[i]; }
9     }
10    return z;
11 }
```

5.3 Hashing de Strings (Polinomial)

Complejidad de Tiempo:

- Precomputo: $\mathcal{O}(N)$
- Consulta de hash de subcadena: $\mathcal{O}(1)$

Complejidad de Espacio: $\mathcal{O}(N)$

Permite comparar subcadenas en $\mathcal{O}(1)$ tras el precomputo,

útil para detectar duplicados o hacer búsqueda binaria sobre la longitud del prefijo/subcadena común. Se recomienda usar doble hashing (dos módulos) para evitar colisiones adversarias.

```
1 struct StringHash {
2     vector<ll> h, p;
3     ll MOD, BASE;
4     StringHash(const string& s, ll mod = 1e9+7, ll base =
5         131) : MOD(mod), BASE(base) {
6         int n = s.size();
7         h.assign(n + 1, 0); p.assign(n + 1, 1);
8         for (int i = 0; i < n; i++) {
9             h[i+1] = (h[i] * BASE + s[i]) % MOD;
10            p[i+1] = (p[i] * BASE) % MOD;
11        }
12    }
13    // Hash del rango [l, r] (0-indexado, inclusivo)
14    ll getHash(int l, int r) {
15        ll res = (h[r+1] - (h[l] * p[r-l+1]) % MOD + MOD *
16            MOD) % MOD;
17        return res;
18    }
19 }
```

5.4 Manacher (Palíndromos)

Complejidad de Tiempo: $\mathcal{O}(N)$

Complejidad de Espacio: $\mathcal{O}(N)$

Encuentra el radio del palíndromo más largo centrado en cada posición, manejando de forma unificada los casos de longitud par e impar mediante la inserción de separadores.

```
1 // Retorna, para la cadena transformada t = "#a#b#a#",
2 // el radio de palindromo en cada centro
3 vector<int> manacher(const string& s) {
4     string t = "#";
5     for (char c : s) { t += c; t += '#'; }
6     int n = t.size();
7     vector<int> radio(n, 0);
8     int l = 0, r = -1;
9     for (int i = 0; i < n; i++) {
10        int k = (i > r) ? 1 : min(radio[l + r - i], r - i
11            + 1);
12        while (i - k >= 0 && i + k < n && t[i-k] == t[i+k]) k++;
13        radio[i] = k--;
14        if (i + k > r) { l = i - k; r = i + k; }
15    }
16    return radio; // longitud del palindromo centrado en i
17 // es radio[i]-1
18 }
```

5.5 Boyer-Moore (Búsqueda de Patrones)

Complejidad de Tiempo: $\mathcal{O}(N+M)$ en promedio, y en el mejor caso llega a ser *sublineal* ($\mathcal{O}(N/M)$) ya que permite saltar varias posiciones del texto de una vez. Peor caso $\mathcal{O}(N \cdot M)$ sin la heurística de sufijo bueno (poco común en la práctica).

Complejidad de Espacio: $\mathcal{O}(M + \Sigma)$, donde Σ es el tamaño del alfabeto.

A diferencia de KMP (que avanza de izquierda a derecha comparando), Boyer-Moore compara el patrón contra el texto **de derecha a izquierda**, y cuando encuentra un mismatch usa la heurística del "*bad character*" para saltar el

patrón varias posiciones de una sola vez, evitando comparaciones innecesarias. Es el algoritmo que usan en la práctica muchos motores de búsqueda de texto (ej. `grep`) por su gran rendimiento en alfabetos grandes (texto natural).

```

1 // Heurística del "bad character": para cada caracter del
  // alfabeto, guarda
2 // la última posición en la que aparece dentro del patrón
3 vector<int> build_bad_char_table(const string& pat, int
  alphabetSize = 256) {
4     vector<int> lastOcc(alphabetSize, -1);
5     for (int i = 0; i < (int)pat.size(); i++) lastOcc[(
  unsigned char)pat[i]] = i;
6     return lastOcc;
7 }
8 // Devuelve las posiciones (0-indexadas) en 'texto' donde
  // inicia 'pat'
9 vector<int> boyer_moore_search(const string& texto, const
  string& pat) {
10     int n = texto.size(), m = pat.size();
11     if (m == 0 || m > n) return {};
12     vector<int> badChar = build_bad_char_table(pat);
13     vector<int> ocurrencias;
14     int s = 0; // desplazamiento del patrón respecto al
  // inicio del texto
15     while (s <= n - m) {
16         int j = m - 1;
17         // Compara de derecha a izquierda mientras los
  // caracteres coincidan
18         while (j >= 0 && pat[j] == texto[s + j]) j--;
19         if (j < 0) {
20             ocurrencias.push_back(s); // match completo
  // encontrado
21             // Desplaza usando el carácter siguiente al
  // patrón (o 1 si no hay más texto)
22             s += (s + m < n) ? m - badChar[(unsigned char)
  texto[s + m]] : 1;
23         } else {
24             // Salta según la última ocurrencia del
  // carácter conflictivo en el patrón
25             int shift = j - badChar[(unsigned char)texto[s
  + j]];
26             s += max(1, shift); // al menos avanza 1 para
  // garantizar progreso
27         }
28     }
29     return ocurrencias;
30 }

```

Variante: Boyer-Moore Voting Algorithm (Elemento Mayoritario)

Complejidad de Tiempo: $\mathcal{O}(N)$

Complejidad de Espacio: $\mathcal{O}(1)$

No confundir con el algoritmo anterior: comparte el nombre del mismo autor (Robert S. Boyer) pero resuelve un problema distinto — encontrar el elemento que aparece **más de $N/2$ veces** en un arreglo, en una sola pasada y sin memoria extra.

```

1 // Encuentra el candidato a elemento mayoritario (aparece
  // > n/2 veces)
2 // IMPORTANTE: requiere una segunda pasada para confirmar
  // que realmente es mayoría,
3 // ya que si no existe tal elemento, el algoritmo igual
  // retorna un candidato inválido
4 int boyer_moore_voting(const vector<int>& arr) {
5     int candidato = 0, contador = 0;
6     for (int x : arr) {
7         if (contador == 0) candidato = x;
8         contador += (x == candidato) ? 1 : -1;
9     }
10    // Verificación (obligatoria)
11    int freq = count(arr.begin(), arr.end(), candidato);
12    return (freq > (int)arr.size() / 2) ? candidato : -1;
13    // -1 si no hay mayoría
14 }

```

6. Programación Dinámica (DP)

6.1 ¿Cuándo Usar Programación Dinámica?

DP aplica cuando el problema cumple **ambas** condiciones:

- **Subestructura óptima:** la solución óptima del problema completo se puede construir a partir de soluciones óptimas de subproblemas más pequeños.
- **Subproblemas superpuestos:** al resolver recursivamente, los mismos subproblemas se repiten muchas veces (si no se repiten, es simple recursión/divide y vencerás, no DP).

Pistas típicas en el enunciado:

- "Cuenta el número de formas de..." / "¿Cuántas maneras hay de...?"
- "Encuentra el mínimo/máximo costo/valor/longitud para..."
- "¿Es posible lograr...?" (decisión, sí/no) con restricciones acumulativas.
- El problema se puede describir con un estado que depende de "decisiones anteriores" (ej. posición actual, capacidad restante, elementos ya usados).
- Fuerza bruta recursiva sería exponencial, pero el número de **estados distintos** es polinomial.

Cómo diseñar la solución (pasos sugeridos):

1. Define el **estado**: ¿qué información mínima necesito para tomar la siguiente decisión? (ej. índice actual, peso restante, máscara de visitados).
2. Define la **transición**: dado un estado, ¿cómo se llega a los estados siguientes/anteriores?
3. Define el/los **caso(s) base**.
4. Decide el **orden de cálculo**: top-down (memoización, más fácil de escribir) o bottom-up (tabulación, evita stack overflow y suele ser más rápido).
5. Verifica la **complejidad**: (cantidad de estados) $\times$ (costo por transición).

6.2 Plantilla General de DP (Top-Down / Memoización)

Complejidad de Tiempo: (número de estados) $\times$ (costo de la transición).

Complejidad de Espacio: $\mathcal{O}$ (número de estados) para la tabla de memo.

Esqueleto reutilizable: solo hay que cambiar el caso base y la transición según el problema. El ejemplo concreto de la transición cuenta caminos en una grilla moviéndose solo hacia la derecha o hacia abajo (resultado conocido: $\binom{i+j}{i}$).

```
1 // ---- Version con arreglo (mas rapida, requiere conocer
  los limites del estado) ----
2 const int MAXI = 1005, MAXJ = 1005;
3 ll dpGeneral[MAXI][MAXJ];
4 bool visitadoDp[MAXI][MAXJ]; // IMPORTANTE: resetear entre
  casos de prueba (memset o fill)
5
6 ll solve(int i, int j /*, parametros adicionales del
  problema */) {
7     // 1) Caso(s) base
8     if (i < 0 || j < 0) return 0;
9     if (i == 0 && j == 0) return 1;
10    // 2) Memo hit: si ya se calculo este estado,
      devolverlo sin recalcularlo
11    if (visitadoDp[i][j]) return dpGeneral[i][j];
12    visitadoDp[i][j] = true;
13    // 3) Transicion: combinar resultados de subproblemas
      mas pequenos
14    ll res = 0;
15    res += solve(i - 1, j);
16    res += solve(i, j - 1);
17    return dpGeneral[i][j] = res % MOD;
18 }
19
20 // ---- Version con map/unordered_map (cuando el estado no
  cabe en un arreglo,
21 // ej. bitmask + nodo, o el rango del estado es muy
  grande/disperso) ----
22 map<pair<int,int>, ll> memoGenerico;
23 ll solveMap(int i, int j) {
24     if (i < 0 || j < 0) return 0;
25     if (i == 0 && j == 0) return 1;
26     auto key = make_pair(i, j);
27     auto it = memoGenerico.find(key);
28     if (it != memoGenerico.end()) return it->second;
29     ll res = solveMap(i - 1, j) + solveMap(i, j - 1);
30     return memoGenerico[key] = res % MOD;
31 }
```

Tip: si la version recursiva (top-down) hace stack overflow por demasiada profundidad, convertir a *bottom-up* (tabulación): iterar los estados en el orden correcto (de los casos base hacia el estado final) usando bucles **for** en vez de recursión.

6.3 Fibonacci con Memoización

Ejemplo clásico de subproblemas superpuestos: $F(n) = F(n - 1) + F(n - 2)$ recalcula los mismos valores exponencialmente sin memoizar. Ver también la sección "Fibonacci Matricial" en el capítulo de Fórmulas Matemáticas Esenciales para la versión $\mathcal{O}(\log n)$.

6.3.1 Versión con Arreglo

Complejidad: $\mathcal{O}(N)$ tiempo y espacio. Ideal cuando $N \lesssim 10^6$ y se conoce el límite de antemano.

```
1 const int MAXN = 1000006;
2 ll memoFib[MAXN];
3 bool calculadoFib[MAXN];
4 ll fibonacci_memo(int n) {
5     if (n <= 1) return n;
6     if (calculadoFib[n]) return memoFib[n];
7     calculadoFib[n] = true;
8     return memoFib[n] = (fibonacci_memo(n - 1) +
9         fibonacci_memo(n - 2)) % MOD;
10 }
```

6.3.2 Versión con unordered_map

Complejidad: $\mathcal{O}(N)$ tiempo, sin necesidad de reservar un arreglo de tamaño fijo por adelantado.

```

1 unordered_map<ll, ll> memoFibMap;
2 ll fibonacci_memo_map(ll n) {
3     if (n <= 1) return n;
4     auto it = memoFibMap.find(n);
5     if (it != memoFibMap.end()) return it->second;
6     ll res = (fibonacci_memo_map(n - 1) +
7     fibonacci_memo_map(n - 2)) % MOD;
8     return memoFibMap[n] = res;
9 }

```

6.3.3 Fast Doubling con Memoización

Complejidad: $\mathcal{O}(\log N)$. La mejor opción para N gigante (hasta 10^{18}) manteniendo un estilo recursivo/memoizado. Se basa en las identidades:

$$F(2k) = F(k) \cdot (2F(k+1) - F(k)), \quad F(2k+1) = F(k)^2 + F(k+1)^2$$

```

1 unordered_map<ll, pair<ll, ll>> memoFastDouble; // n -> {F(n), F(n+1)}
2 pair<ll, ll> fib_fast_doubling(ll n) {
3     if (n == 0) return {0, 1};
4     auto it = memoFastDouble.find(n);
5     if (it != memoFastDouble.end()) return it->second;
6     auto [fk, fk1] = fib_fast_doubling(n / 2);
7     ll c = (fk * ((2 * fk1 - fk % MOD + MOD) % MOD)) % MOD; // F(2k)
8     ll d = (fk * fk % MOD + fk1 * fk1 % MOD) % MOD; // F(2k+1)
9     pair<ll, ll> res;
10    if (n % 2 == 0) res = {c, d};
11    else res = {d, (c + d) % MOD};
12    return memoFastDouble[n] = res;
13 }
14 ll fibonacci_fast_doubling(ll n) { return
    fib_fast_doubling(n).first; }

```

6.4 Mochila 0-1 (0-1 Knapsack)

Complejidad de Tiempo: $\mathcal{O}(N \cdot W)$, donde N es la cantidad de elementos y W la capacidad de peso máximo de la mochila.

Complejidad de Espacio: $\mathcal{O}(W)$ mediante optimización con un arreglo unidimensional.

```

1 int knapsack(int W, const vector<int>& peso, const vector<
    int>& valor, int n) {
2     vector<int> dp(W + 1, 0);
3     for (int i = 0; i < n; i++) {
4         for (int w = W; w >= peso[i]; w--) {
5             dp[w] = max(dp[w], dp[w - peso[i]] + valor[i]);
6         }
7     }
8     return dp[W];
9 }

```

6.5 Longest Common Subsequence (LCS)

Complejidad de Tiempo: $\mathcal{O}(|S| \cdot |T|)$

Complejidad de Espacio: $\mathcal{O}(|S| \cdot |T|)$

Encuentra la subsecuencia común más larga entre dos strings S y T .

```

1 int lcs(string S, string T) {
2     int n = S.size(), m = T.size();
3     vector<vector<int>>> dp(n + 1, vector<int>(m + 1, 0));
4     for (int i = 1; i <= n; i++) {
5         for (int j = 1; j <= m; j++) {

```

```

6         if (S[i-1] == T[j-1]) dp[i][j] = dp[i-1][j-1]
7         + 1;
8         else dp[i][j] = max(dp[i-1][j], dp[i][j-1]);
9     }
10    return dp[n][m];
11 }

```

6.6 Longest Increasing Subsequence (LIS)

Complejidad de Tiempo: $\mathcal{O}(N \log N)$ mediante búsqueda binaria de reemplazo.

Complejidad de Espacio: $\mathcal{O}(N)$

Calcula la longitud de la subsecuencia estrictamente creciente más larga de un arreglo de tamaño N .

```

1 int lis(const vector<int>& a) {
2     vector<int> tails;
3     for (int x : a) {
4         auto it = lower_bound(tails.begin(), tails.end(),
5         x);
6         if (it == tails.end()) tails.push_back(x);
7         else *it = x;
8     }
9     return tails.size();
10 }

```

6.7 DP con Máscaras de Bits (Bitmask DP - TSP)

Complejidad de Tiempo: $\mathcal{O}(2^N \cdot N^2)$

Complejidad de Espacio: $\mathcal{O}(2^N \cdot N)$

Resuelve el Problema del Viajero (TSP) encontrando el ciclo hamiltoniano de coste mínimo en un grafo con $N \leq 20$. El estado se representa combinando un bitmask y el nodo actual.

```

1 int n; int dist[20][20]; int dp[1 << 20][20]; //
    Inicializar en -1
2 int tsp(int mask, int u) {
3     if (mask == (1 << n) - 1) return dist[u][0];
4     if (dp[mask][u] != -1) return dp[mask][u];
5     int ans = 1e9;
6     for (int v = 0; v < n; v++) {
7         if (!(mask & (1 << v))) {
8             ans = min(ans, dist[u][v] + tsp(mask | (1 << v), v));
9         }
10    }
11    return dp[mask][u] = ans;
12 }

```

6.8 DP en Árboles (Tree DP)

Complejidad de Tiempo: $\mathcal{O}(V)$ (o $\mathcal{O}(V \log V)$ si se combinan estructuras auxiliares por nodo).

Complejidad de Espacio: $\mathcal{O}(V)$

Patrón general para calcular información agregada en árboles mediante DFS post-order: primero se resuelven los hijos, luego se combina el resultado en el padre. El ejemplo calcula el tamaño del subárbol y el conjunto independiente máximo ponderado.

```

1 vector<vector<int>>> adj_tree;
2 vector<ll> peso_nodo;

```

```
3 11 dpIncluye[200005], dpExcluye[200005]; // Inicializar en
   0
4 void tree_dp(int u, int padre) {
5     dpIncluye[u] = peso_nodo[u];
6     dpExcluye[u] = 0;
7     for (int v : adj_tree[u]) {
8         if (v == padre) continue;
9         tree_dp(v, u);
10        dpIncluye[u] += dpExcluye[v]; // Si tomo u, no
        puedo tomar sus hijos
11        dpExcluye[u] += max(dpIncluye[v], dpExcluye[v]);
12    }
13 }
14 // Respuesta final: max(dpIncluye[raiz], dpExcluye[raiz])
```

7. Manipulación de Bits (Bitwise)

7.1 Operaciones Básicas y Trucos Fundamentales

Complejidad de Tiempo: $\mathcal{O}(1)$ por operación

Los operadores $\&$ (AND), $|$ (OR), $\wedge$ (XOR), $\sim$ (NOT), $\ll$ y $\gg$ (shifts) trabajan directamente sobre la representación binaria. Estos son los patrones que más se repiten en cursos.

```
1 int x = 0;
2 // Consultar el bit i (0-indexado desde el menos
  significativo)
3 bool bit_i = (x >> i) & 1;
4 // Prender (set) el bit i
5 x |= (1 << i);
6 // Apagar (clear) el bit i
7 x &= ~(1 << i);
8 // Invertir (toggle) el bit i
9 x ^= (1 << i);
10 // Obtener el bit menos significativo prendido (lowest set
    bit)
11 int lsb = x & (-x);
12 // Apagar el bit menos significativo prendido
13 int sin_lsb = x & (x - 1);
14 // Verificar si x es potencia de 2 (y distinto de 0)
15 bool es_potencia_2 = x > 0 && (x & (x - 1)) == 0;
16 // Redondear hacia arriba a la siguiente potencia de 2
17 ll siguiente_potencia_2(ll n) {
18     ll p = 1;
19     while (p < n) p <<= 1;
20     return p;
21 }
```

7.2 Conteo de Bits (Popcount)

Complejidad de Tiempo: $\mathcal{O}(1)$ amortizado con funciones `__builtin` (usan instrucciones nativas del procesador), $\mathcal{O}(\log N)$ con la versión manual.

Contar cuántos bits están prendidos en un número es una operación extremadamente común. GCC ofrece funciones nativas muy rápidas que conviene memorizar.

```
1 int x = 42;
2 int cnt = __builtin_popcount(x); // cuenta bits en
  1 (int, 32 bits)
3 int cntll = __builtin_popcountll(x); // version para
  long long (64 bits)
4 int trailing = __builtin_ctz(x); // cantidad de
  ceros al final (trailing zeros)
5 int leading = __builtin_clz(x); // cantidad de
  ceros al inicio (leading zeros)
6 int highBit = 31 - __builtin_clz(x); // indice del bit
  mas significativo prendido
7 // Version manual (por si no se puede usar __builtin, ej.
  lenguajes sin soporte)
8 int popcount_manual(unsigned int x) {
9     int cnt = 0;
10    while (x) { cnt += x & 1; x >>= 1; }
11    return cnt;
12 }
13 // Suma total de bits prendidos en todos los numeros de 1
  a N
14 ll suma_bits_1_a_n(ll n) {
15     ll total = 0;
16     for (int bit = 0; (1LL << bit) <= n; bit++) {
17         ll ciclo = 1LL << (bit + 1);
18         ll completos = (n + 1) / ciclo;
19         total += completos * (1LL << bit);
20         ll resto = (n + 1) % ciclo;
21         total += max(0LL, resto - (1LL << bit));
22     }
23     return total;
24 }
```

Tabla de Popcount para Todos los Números en $[0, N]$

Complejidad de Tiempo: $\mathcal{O}(N)$ para construir la tabla completa.

Complejidad de Espacio: $\mathcal{O}(N)$

A diferencia de `suma_bits_1_a_n` (que da un único total acumulado), esta tabla guarda el **popcount individual** de cada número de 0 a N , permitiendo consultas repetidas en $\mathcal{O}(1)$. Se construye con programación dinámica: el popcount de i es igual al popcount de i sin su bit menos significativo, más ese bit. Es la forma estándar de precalcular popcounts masivos sin pagar $\mathcal{O}(\log N)$ por cada consulta individual (o cuando no se puede usar `__builtin_popcount`, ej. jueces que no usan GCC).

```
1 // cnt[i] = cantidad de bits en 1 en la representacion
  binaria de i, para 0 <= i <= n
2 vector<int> popcount_table(int n) {
3     vector<int> cnt(n + 1, 0);
4     for (int i = 1; i <= n; i++) {
5         cnt[i] = cnt[i >> 1] + (i & 1); // i >> 1 quita el
        bit menos significativo
5     }
6     return cnt;
7 }
8 // Variante equivalente usando i & (i-1) (apaga el lowest
  set bit)
9 vector<int> popcount_table_v2(int n) {
10    vector<int> cnt(n + 1, 0);
11    for (int i = 1; i <= n; i++) {
12        cnt[i] = cnt[i & (i - 1)] + 1;
13    }
14    return cnt;
15 }
16 }
```

7.3 Suma y Resta Binaria (Bit a Bit)

Complejidad de Tiempo: $\mathcal{O}(\log N)$

Implementación manual de suma/resta usando solo operadores bitwise, útil para entender cómo el hardware realiza la operación (o cuando se trabaja con aritmética modular sobre bits, como en criptografía básica). El truco clave: $a \hat{b}$ da la suma sin acarreo, y $(a \& b) \ll 1$ da el acarreo a propagar.

```
1 // Suma de dos enteros usando solo operadores bitwise
2 int suma_bitwise(int a, int b) {
3     while (b != 0) {
4         int carry = (a & b) << 1; // acarreo: bits donde
        ambos son 1, desplazado
5         a = a ^ b; // suma sin acarreo
6         b = carry;
7     }
8     return a;
9 }
10 // Resta usando complemento a dos: a - b = a + (~b + 1)
11 int resta_bitwise(int a, int b) {
12     return suma_bitwise(a, suma_bitwise(~b, 1));
13 }
```

7.4 Multiplicación Binaria (Russian Peasant / Egyptian Multiplication)

Complejidad de Tiempo: $\mathcal{O}(\log N)$

Multiplica dos números usando solo sumas y desplazamientos.

tos de bits, descomponiendo un factor en su representación binaria. Es la misma idea que la exponenciación rápida pero aplicada a la multiplicación, y es clave para hacer **multiplicación modular segura** cuando $a \cdot b$ podría desbordar incluso a long long (ej. módulos cercanos a 10^{18}).

```
1 // Multiplica a*b de forma segura modulo 'mod', evitando
  overflow (sin usar __int128)
2 ll mulmod_binario(ll a, ll b, ll mod) {
3     ll result = 0;
4     a %= mod;
5     while (b > 0) {
6         if (b & 1) result = (result + a) % mod; // si el
          bit actual de b esta prendido, suma
7         a = (a + a) % mod; // duplica a (equivalente a a
          << 1)
8         b >>= 1;
9     }
10    return result;
11 }
12 // Multiplicacion binaria simple sin modulo (solo shifts y
  sumas)
13 ll multiplicar_binario(ll a, ll b) {
14     ll result = 0;
15     while (b > 0) {
16         if (b & 1) result += a;
17         a <<= 1;
18         b >>= 1;
19     }
20    return result;
21 }
```

7.5 Enumeración de Submáscaras (Submask Enumeration)

Complejidad de Tiempo: $\mathcal{O}(3^N)$ para enumerar todas las submáscaras de todas las máscaras de N bits (no $\mathcal{O}(4^N)$, es una cota más ajustada por el número de pares (mask, sub-mask)).

Técnica clásica para recorrer todos los subconjuntos de un conjunto representado como bitmask, muy usada en DP sobre subconjuntos (*SOS DP* - Sum over Subsets).

```
1 // Recorre todas las submascaras de 'mask' (incluyendo la
  mascara vacia y mask mismo)
2 void enumerar_submascaras(int mask) {
3     for (int sub = mask; ; sub = (sub - 1) & mask) {
4         // procesar 'sub' aqui
5         if (sub == 0) break;
6     }
7 }
8 // SOS DP: para cada mascara, suma los valores de todas
  sus submascaras
9 // dp[mask] empieza como el valor base de esa mascara
  exacta
10 void sos_dp(vector<ll>& dp, int N_BITS) {
11     for (int bit = 0; bit < N_BITS; bit++) {
12         for (int mask = 0; mask < (1 << N_BITS); mask++) {
13             if (mask & (1 << bit)) dp[mask] += dp[mask ^
14             (1 << bit)];
15         }
16 }
```

7.6 Propiedades y Trucos con XOR

Complejidad de Tiempo: $\mathcal{O}(1)$ a $\mathcal{O}(N)$ según el truco
XOR es asociativo, conmutativo y $x \oplus x = 0$, $x \oplus 0 = x$. Estas propiedades permiten resolver varios problemas clásicos sin memoria extra.

```
1 // Encontrar el UNICO numero que no se repite, cuando
  todos los demas aparecen 2 veces
```

```
2 int unico_no_repetido(const vector<int>& arr) {
3     int res = 0;
4     for (int x : arr) res ^= x;
5     return res;
6 }
7 // Intercambiar dos variables sin variable auxiliar
8 void swap_xor(int& a, int& b) {
9     if (&a == &b) return; // cuidado: falla si a y b son
        la misma direccion
10    a ^= b; b ^= a; a ^= b;
11 }
12 // Prefijos XOR: permite calcular XOR de cualquier rango [
    1, r] en  $\mathcal{O}(1)$ 
13 vector<int> prefix_xor(const vector<int>& arr) {
14     vector<int> pre(arr.size() + 1, 0);
15     for (int i = 0; i < (int)arr.size(); i++) pre[i+1] =
        pre[i] ^ arr[i];
16     return pre;
17 }
18 int xor_rango(const vector<int>& pre, int l, int r) { //
    0-indexado, inclusivo
19     return pre[r+1] ^ pre[l];
20 }
21 // Encontrar los DOS unicos numeros que no se repiten (los
  demas aparecen 2 veces)
22 pair<int,int> dos_unicos_no_repetidos(const vector<int>&
  arr) {
23     int xorTotal = 0;
24     for (int x : arr) xorTotal ^= x;
25     int diffBit = xorTotal & (-xorTotal); // un bit donde
        los dos unicos difieren
26     int a = 0, b = 0;
27     for (int x : arr) {
28         if (x & diffBit) a ^= x; else b ^= x;
29     }
30     return {a, b};
31 }
```

7.7 std::bitset: Uso y Conversión

Complejidad de Tiempo: $\mathcal{O}(N/64)$ por operación bitwise completa (AND, OR, XOR, shift) sobre todo el bitset, gracias al paralelismo a nivel de palabra del procesador — esto es lo que lo hace mucho más rápido que un `vector<bool>` o un arreglo de bools para operaciones masivas.

`std::bitset<N>` representa una secuencia **fija** de N bits (N debe ser una constante conocida en tiempo de compilación). Es ideal para acelerar DP's booleanos, cribas, y cualquier situación con muchas operaciones bit a bit sobre conjuntos grandes.

```
1 #include <bitset>
2 bitset<32> b1; //
    00000000000000000000000000000000, todo en 0
3 bitset<32> b2(42); // inicializado desde un
    entero
4 bitset<8> b3("10110010"); // inicializado desde un
    string binario
5 // Operaciones basicas
6 b1[3] = 1; // prender el bit 3 (operator[])
7 b1.set(5); // prender el bit 5
8 b1.reset(3); // apagar el bit 3
9 b1.flip(0); // invertir el bit 0
10 bool bit5 = b1.test(5); // consultar el bit 5 (mas
    seguro que operator[])
11 int cantidadPrendidos = b1.count(); // cuenta bits en 1,
    equivalente a popcount
12 bool hayAlguno = b1.any(); // true si algun bit
    esta prendido
13 bool noHayNinguno = b1.none(); // true si todos
    estan en 0
14 bool todosPrendidos = b1.all(); // true si todos
    estan en 1
15 size_t tamano = b1.size(); // tamano fijo N (32
    en este caso)
16 // Operadores bitwise funcionan igual que con enteros,
    pero sobre todo el bitset
17 bitset<32> b4 = b1 & b2;
18 bitset<32> b5 = b1 | b2;
19 bitset<32> b6 = b1 ^ b2;
```

```

20 b1 <<= 3; // shift a la izquierda
21 b1 >>= 2; // shift a la derecha

```

Conversión a String y a Entero

```

1 bitset<8> b("10110010");
2 // bitset -> string
3 string s = b.to_string(); // "10110010"
4 // bitset -> entero (cuidado con el tamaño: to_ulong lanza
5 // excepción si no cabe en unsigned long)
6 unsigned long numUL = b.to_ulong(); // para bitsets
7 // pequeños (<=32 bits usualmente)
8 unsigned long long numULL = b.to_ullong(); // para bitsets
9 // de hasta 64 bits
10 // entero -> bitset (constructor directo)
11 int x = 178;
12 bitset<8> desdeEntero(x); // 10110010
13 // string -> bitset (constructor directo)
14 string binStr = "10110010";
15 bitset<8> desdeString(binStr);
16 // También se puede construir desde un string con
17 // caracteres personalizados
18 // (por ejemplo, usando '.'/'#' en vez de '0'/'1', útil
19 // para leer mapas de grilla)
20 string grid = "##..#.#.";
21 bitset<8> desdeGrid(grid, 0, string::npos, '.', '#'); //
22 // '.' -> 0, '#' -> 1

```

7.8 Casos de Uso de Bitmasking

Complejidad de Tiempo: varía según el caso, pero comparte el patrón de representar un subconjunto o estado con los bits de un entero.

Además del DP con bitmask ya visto (TSP), estos son los usos más frecuentes en competencia:

Representar Visitados / Subconjuntos en Grafos Pequeños

Cuando $N \leq 20$ -22, un entero puede representar qué nodos ya se visitaron, permitiendo DP sobre subconjuntos como estado.

```

1 // Contar caminos hamiltonianos que empiezan en el nodo 0,
2 // en un grafo de N nodos
3 int N; bool adj[20][20];
4 ll dp[1 << 20][20]; // dp[mask][u] = caminos que visitan
5 // exactamente 'mask' y terminan en u
6 ll contar_caminos_hamiltonianos() {
7     memset(dp, 0, sizeof(dp));
8     dp[1][0] = 1; // solo se visito el nodo 0
9     for (int mask = 1; mask < (1 << N); mask++) {
10         for (int u = 0; u < N; u++) {
11             if (!(mask & (1 << u)) || dp[mask][u] == 0)
12                 continue;
13             for (int v = 0; v < N; v++) {
14                 if (adj[u][v] && !(mask & (1 << v))) {
15                     dp[mask | (1 << v)][v] += dp[mask][u];
16                 }
17             }
18         }
19     }
20     ll total = 0;
21     for (int u = 0; u < N; u++) total += dp[(1 << N) - 1][u];
22     return total;
23 }

```

Problema de Asignación con Bitmask (Assignment Problem)

Asignar N trabajadores a N tareas minimizando el costo, cuando N es pequeño (≤ 20). El bitmask representa qué tareas ya fueron asignadas.

```

1 int N; int costo[20][20];
2 int dpAsig[1 << 20]; // Inicializar en -1
3 // dp(mask) = costo mínimo de asignar las tareas pendientes
4 // en 'mask' a los primeros popcount(mask) trabajadores
5 int asignacion_optima(int mask) {
6     int trabajador = __builtin_popcount(mask);
7     if (trabajador == N) return 0;
8     if (dpAsig[mask] != -1) return dpAsig[mask];
9     int mejor = INT_MAX;
10     for (int tarea = 0; tarea < N; tarea++) {
11         if (!(mask & (1 << tarea))) {
12             mejor = min(mejor, costo[trabajador][tarea] +
13                 asignacion_optima(mask | (1 << tarea)));
14         }
15     }
16     dpAsig[mask] = mejor;
17 }

```

Subset Sum Acelerado con bitset

Acelera el clásico DP de subset sum de $\mathcal{O}(N \cdot W)$ a $\mathcal{O}(N \cdot W/64)$ representando el conjunto de sumas alcanzables como un bitset, donde el bit i indica si la suma i es alcanzable.

```

1 const int MAXW = 100005;
2 bitset<MAXW> subset_sum_alcanzable(const vector<int>&
3     pesos) {
4     bitset<MAXW> dp;
5     dp[0] = 1; // suma 0 siempre es alcanzable (no
6     // eligiendo nada)
7     for (int w : pesos) {
8         dp |= (dp << w); // para cada suma ya alcanzable s
9         // , ahora s+w también lo es
10     }
11     return dp; // dp[x] == 1 si existe un subconjunto que
12     // suma exactamente x
13 }

```

Contar Pares/Tripletas con Propiedades sobre Bits

Cuando se necesita saber, para cada máscara, cuántos elementos del arreglo tienen exactamente esos bits (o son submáscara/supermáscara de ella), se usa un arreglo indexado por máscara en vez de recorrer todo el arreglo cada vez.

```

1 // cnt[mask] = cantidad de elementos del arreglo original
2 // iguales a 'mask'
3 // Tras aplicar SOS DP (ver sección anterior), cnt[mask]
4 // pasa a significar
5 // "cantidad de elementos que son SUBMASCARA de mask"
6 vector<ll> contar_por_mascara(const vector<int>& arr, int
7     bits) {
8     vector<ll> cnt(1 << bits, 0);
9     for (int x : arr) cnt[x]++;
10     for (int bit = 0; bit < bits; bit++)
11         for (int mask = 0; mask < (1 << bits); mask++)
12             if (mask & (1 << bit)) cnt[mask] += cnt[mask ^
13                 (1 << bit)];
14     return cnt;
15 }

```

8. Matemáticas Teóricas

8.1 MCD (Máximo Común Divisor) y MCM (Mínimo Común Múltiplo)

Complejidad de Tiempo: $\mathcal{O}(\log(\min(a, b)))$

Complejidad de Espacio: $\mathcal{O}(1)$ utilizando la función recursiva nativa o el algoritmo de Euclides.

Implementación básica de GCD utilizando recursión y cálculo directo de LCM basándose en la identidad $a \cdot b = \gcd(a, b) \cdot \text{lcm}(a, b)$.

```
1 // Nota: En C++17 existe std::gcd y std::lcm en la
  libreria <numeric>
2 long long gcd(long long a, long long b) {
3     return b == 0 ? a : gcd(b, a % b);
4 }
5 long long lcm(long long a, long long b) {
6     if (a == 0 || b == 0) return 0;
7     return (a / gcd(a, b)) * b; // Evita el desbordamiento
  de enteros (overflow)
8 }
```

8.2 Algoritmo de Euclides Extendido

Complejidad de Tiempo: $\mathcal{O}(\log(\min(a, b)))$

Complejidad de Espacio: $\mathcal{O}(\log(\min(a, b)))$ debido a la pila de recursión.

Halla los coeficientes enteros x, y de la identidad de Bézout para resolver la ecuación lineal $ax + by = \gcd(a, b)$, lo cual sirve también para hallar el inverso multiplicativo modular.

```
1 ll extgcd(ll a, ll b, ll &x, ll &y) {
2     if (b == 0) { x = 1; y = 0; return a; }
3     ll x1, y1;
4     ll d = extgcd(b, a % b, x1, y1);
5     x = y1; y = x1 - y1 * (a / b);
6     return d;
7 }
```

8.3 Criba de Eratóstenes (Sieve)

Complejidad de Tiempo: $\mathcal{O}(N \log \log N)$

Complejidad de Espacio: $\mathcal{O}(N)$

Marca todos los números primos hasta N . La variante mostrada además calcula el menor factor primo (*smallest prime factor*) de cada número, lo cual permite factorizar cualquier valor en $\mathcal{O}(\log N)$.

```
1 const int MAXN = 1000006;
2 vector<int> spf(MAXN); // menor factor primo de cada
  numero
3 vector<int> primos;
4 void sieve() {
5     for (int i = 2; i < MAXN; i++) {
6         if (spf[i] == 0) {
7             spf[i] = i;
8             primos.push_back(i);
9         }
10        for (int p : primos) {
11            if (p > spf[i] || (ll)i * p >= MAXN) break;
12            spf[i * p] = p;
13        }
14    }
15 }
16 // Factoriza n usando el sieve precalculado
```

```
17 vector<int> factorizar(int n) {
18     vector<int> factores;
19     while (n > 1) { factores.push_back(spf[n]); n /= spf[n]; }
20     return factores;
21 }
```

8.4 Exponenciación Rápida (Fast/Modular Power)

Complejidad de Tiempo: $\mathcal{O}(\log E)$, donde E es el exponente.

Complejidad de Espacio: $\mathcal{O}(1)$ (versión iterativa)

Calcula $\text{base}^{\text{exp}} \bmod m$ eficientemente mediante exponenciación binaria, evitando el cálculo directo de potencias enormes.

```
1 ll modpow(ll base, ll exp, ll mod) {
2     ll result = 1;
3     base %= mod;
4     while (exp > 0) {
5         if (exp & 1) result = (result * base) % mod;
6         base = (base * base) % mod;
7         exp >>= 1;
8     }
9     return result;
10 }
```

8.5 Inverso Multiplicativo Modular

Complejidad de Tiempo: $\mathcal{O}(\log MOD)$

Complejidad de Espacio: $\mathcal{O}(1)$

Calcula el inverso de a módulo m (donde m es primo) usando el Pequeño Teorema de Fermat: $a^{-1} \equiv a^{m-2} \pmod{m}$.

```
1 const ll MOD = 1e9 + 7; // debe ser primo
2 ll inv_mod(ll a) {
3     return modpow(a, MOD - 2, MOD);
4 }
```

8.6 Combinatoria: Factoriales y nCr con Módulo

Complejidad de Tiempo: $\mathcal{O}(N \log MOD)$ para el pre-computo, $\mathcal{O}(1)$ por consulta de nCr .

Complejidad de Espacio: $\mathcal{O}(N)$

Precalcula factoriales y sus inversos modulares para responder consultas de combinatoria (nCr , nPr) en tiempo constante.

```
1 const int MAXF = 1000006;
2 vector<ll> fact(MAXF), inv_fact(MAXF);
3 void precalc_factoriales() {
4     fact[0] = 1;
5     for (int i = 1; i < MAXF; i++) fact[i] = (fact[i-1] * i) % MOD;
6     inv_fact[MAXF - 1] = inv_mod(fact[MAXF - 1]);
7     for (int i = MAXF - 2; i >= 0; i--) inv_fact[i] = (inv_fact[i+1] * (i+1)) % MOD;
8 }
9 ll nCr(int n, int r) {
10    if (r < 0 || r > n) return 0;
11    return fact[n] * inv_fact[r] % MOD * inv_fact[n-r] % MOD;
12 }
```

8.7 Test de Primalidad Miller-Rabin

Complejidad de Tiempo: $\mathcal{O}(k \log^3 N)$, donde k es el número de rondas de testigos.

Complejidad de Espacio: $\mathcal{O}(1)$

Test de primalidad probabilístico (determinístico para $N < 3.3 \times 10^{14}$ con los testigos mostrados) mucho más rápido que la división por tentativa para números grandes.

```

1 ll mulmod(ll a, ll b, ll m) { return (__int128)a * b % m;
2 }
3 ll modpow128(ll base, ll exp, ll mod) {
4     ll result = 1; base %= mod;
5     while (exp > 0) {
6         if (exp & 1) result = mulmod(result, base, mod);
7         base = mulmod(base, base, mod);
8         exp >>= 1;
9     }
10    return result;
11 }
12 bool miller_rabin(ll n) {
13     if (n < 2) return false;
14     for (ll p : {2,3,5,7,11,13,17,19,23,29,31,37}) {
15         if (n % p == 0) return n == p;
16     }
17     ll d = n - 1; int r = 0;
18     while (d % 2 == 0) { d /= 2; r++; }
19     for (ll a : {2,3,5,7,11,13,17,19,23,29,31,37}) {
20         if (a >= n) continue;
21         ll x = modpow128(a, d, n);
22         if (x == 1 || x == n - 1) continue;
23         bool compuesto = true;
24         for (int i = 0; i < r - 1; i++) {
25             x = mulmod(x, x, n);
26             if (x == n - 1) { compuesto = false; break; }
27         }
28         if (compuesto) return false;
29     }
30     return true;

```

8.8 Teorema Chino del Resto (CRT) — Implementación

Complejidad de Tiempo: $\mathcal{O}(K \log M)$, donde K es la cantidad de congruencias.

Resuelve el sistema $x \equiv r_i \pmod{m_i}$ combinando las congruencias de dos en dos con Euclides extendido. Funciona incluso si los m_i NO son coprimos entre sí (a diferencia de la formulación clásica), siempre que el sistema sea consistente.

```

1 // Combina x1 (mod m1) y x2 (mod m2) en una
2 // sola congruencia (mod lcm(m1,m2))
3 // Retorna false si el sistema es inconsistente (sin
4 // solucion)
5 bool crt_combinar(ll r1, ll m1, ll r2, ll m2, ll &r, ll &m)
6 {
7     ll x, y;
8     ll g = extgcd(m1, m2, x, y); // reutiliza extgcd de la
9     // seccion anterior (con long long)
10    if ((r2 - r1) % g != 0) return false; // inconsistente
11    ll lcm_ = m1 / g * m2;
12    ll modulo = m2 / g;
13    ll despl = ((r2 - r1) / g % modulo * x % modulo +
14    modulo) % modulo;
15    r = (r1 + m1 * despl) % lcm_;
16    if (r < 0) r += lcm_;
17    m = lcm_;
18    return true;
19 }
20 // Resuelve el sistema completo dado por vectores
21 // paralelos r[] y m[]
22 bool crt(vector<ll>& r, vector<ll>& m, ll &resR, ll &resM)
23 {

```

```

17 resR = 0; resM = 1;
18 for (int i = 0; i < (int)r.size(); i++) {
19     if (!crt_combinar(resR, resM, r[i], m[i], resR,
20     resM)) return false;
21 }
22 return true;

```

8.9 Función de Möbius

Complejidad de Tiempo: $\mathcal{O}(N \log \log N)$ para precalcular todos los valores hasta N (criba lineal).

Usada en técnicas de inclusión-exclusión sobre divisores, como contar pares coprimos.

```

1 const int MAXM = 1000006;
2 vector<int> mu(MAXM);
3 void calcular_mobius() {
4     vector<int> spfLocal(MAXM, 0);
5     vector<int> primosLocal;
6     mu[1] = 1;
7     for (int i = 2; i < MAXM; i++) {
8         if (spfLocal[i] == 0) { spfLocal[i] = i;
9         primosLocal.push_back(i); mu[i] = -1; }
10        for (int p : primosLocal) {
11            if (p > spfLocal[i] || (ll)i * p >= MAXM)
12                break;
13            spfLocal[i * p] = p;
14            mu[i * p] = (p == spfLocal[i]) ? 0 : -mu[i];
15        }
16    }
17    // mu(n) = 0 si n tiene un factor primo al cuadrado
18    // mu(n) = (-1)^k si n es producto de k primos distintos

```

8.10 Fórmula de Legendre (Exponente de un Primo en $N!$)

Complejidad de Tiempo: $\mathcal{O}(\log_p N)$

Calcula cuántas veces divide el primo p a $N!$, sin calcular el factorial directamente. Útil para saber si $N!$ es divisible por algo, o para calcular $N! \pmod{p^k}$.

```

1 // Exponente de p en la factorizacion prima de n!
2 ll legendre_exponente(ll n, ll p) {
3     ll cnt = 0;
4     while (n > 0) { n /= p; cnt += n; }
5     return cnt;
6 }

```

9. Geometría Computacional

9.1 Primitivas Básicas: Punto, Producto Cruz y Punto

Complejidad de Tiempo: $\mathcal{O}(1)$ por operación

Operaciones fundamentales sobre puntos 2D representados como pares (x, y) . El producto cruz determina la orientación relativa de tres puntos (giro izquierda/derecha/colineal).

```
1 struct Point {
2     double x, y;
3     Point operator-(const Point& o) const { return {x - o.x, y - o.y}; }
4 };
5 // Producto cruz de (O->A) x (O->B). Positivo: giro
// antihorario (izquierda)
6 double cross(const Point& O, const Point& A, const Point& B) {
7     return (A.x - O.x) * (B.y - O.y) - (A.y - O.y) * (B.x - O.x);
8 }
9 double dot(const Point& O, const Point& A, const Point& B) {
10     return (A.x - O.x) * (B.x - O.x) + (A.y - O.y) * (B.y - O.y);
11 }
12 double dist(const Point& A, const Point& B) {
13     return hypot(A.x - B.x, A.y - B.y);
14 }
```

9.2 Envolverte Convexa (Convex Hull - Andrew Monotone Chain)

Complejidad de Tiempo: $\mathcal{O}(N \log N)$, dominado por el ordenamiento de puntos.

Complejidad de Espacio: $\mathcal{O}(N)$

Calcula el polígono convexo más pequeño que contiene a todos los puntos dados, construyendo la cadena inferior y superior por separado.

```
1 vector<Point> convex_hull(vector<Point> pts) {
2     int n = pts.size(), k = 0;
3     if (n < 3) return pts;
4     sort(pts.begin(), pts.end(), [](const Point& a, const Point& b) {
5         return a.x < b.x || (a.x == b.x && a.y < b.y);
6     });
7     vector<Point> hull(2 * n);
8     // Cadena inferior
9     for (int i = 0; i < n; i++) {
10         while (k >= 2 && cross(hull[k-2], hull[k-1], pts[i]) <= 0) k--;
11         hull[k++] = pts[i];
12     }
13     // Cadena superior
14     for (int i = n - 2, t = k + 1; i >= 0; i--) {
15         while (k >= t && cross(hull[k-2], hull[k-1], pts[i]) <= 0) k--;
16         hull[k++] = pts[i];
17     }
18     hull.resize(k - 1);
19     return hull;
20 }
```

9.3 Área y Punto Dentro de Polígono

Complejidad de Tiempo: $\mathcal{O}(N)$ para ambas operaciones.
Complejidad de Espacio: $\mathcal{O}(1)$

El área se calcula con la fórmula del *shoelace*. La pertenencia de un punto se determina con el algoritmo de *ray casting*, contando cuántas veces un rayo horizontal cruza los bordes del polígono.

```
1 double poly_area(const vector<Point>& poly) {
2     double area = 0;
3     int n = poly.size();
4     for (int i = 0; i < n; i++) {
5         int j = (i + 1) % n;
6         area += poly[i].x * poly[j].y - poly[j].x * poly[i].y;
7     }
8     return fabs(area) / 2.0;
9 }
10 bool point_in_polygon(const Point& p, const vector<Point>& poly) {
11     int n = poly.size();
12     bool dentro = false;
13     for (int i = 0, j = n - 1; i < n; j = i++) {
14         bool cond = ((poly[i].y > p.y) != (poly[j].y > p.y)) &&
15             (p.x < (poly[j].x - poly[i].x) * (p.y - poly[i].y) / (poly[j].y - poly[i].y) + poly[i].x);
16         if (cond) dentro = !dentro;
17     }
18     return dentro;
19 }
```

9.4 Intersección de Segmentos

Complejidad de Tiempo: $\mathcal{O}(1)$

Determina si dos segmentos de recta se intersectan, cubriendo tanto el caso general (usando orientaciones/producto cruz) como los casos especiales colineales.

```
1 int orientation(const Point& p, const Point& q, const Point& r) {
2     double val = cross(p, q, r);
3     if (fabs(val) < 1e-9) return 0; // colineales
4     return (val > 0) ? 1 : 2; // 1: antihorario, 2: horario
5 }
6 bool on_segment(const Point& p, const Point& q, const Point& r) {
7     return q.x <= max(p.x, r.x) && q.x >= min(p.x, r.x) &&
8         q.y <= max(p.y, r.y) && q.y >= min(p.y, r.y);
9 }
10 bool segments_intersect(const Point& p1, const Point& q1, const Point& p2, const Point& q2) {
11     int o1 = orientation(p1, q1, p2), o2 = orientation(p1, q1, q2);
12     int o3 = orientation(p2, q2, p1), o4 = orientation(p2, q2, q1);
13     if (o1 != o2 && o3 != o4) return true; // caso general
14     if (o1 == 0 && on_segment(p1, p2, q1)) return true;
15     if (o2 == 0 && on_segment(p1, q2, q1)) return true;
16     if (o3 == 0 && on_segment(p2, p1, q2)) return true;
17     if (o4 == 0 && on_segment(p2, q1, q2)) return true;
18     return false;
19 }
```

9.5 Punto de Intersección de Dos Rectas

Complejidad de Tiempo: $\mathcal{O}(1)$

Calcula el punto exacto donde se cruzan dos rectas (no segmentos) definidas cada una por dos puntos. Si las rectas son paralelas, el determinante da cero y no hay solución única.

```
1 // Retorna true si existe interseccion unica; el resultado se guarda en 'out'
2 bool line_intersection(const Point& p1, const Point& p2,
```

```

3         const Point& p3, const Point& p4,
4         Point& out) {
5     double a1 = p2.y - p1.y, b1 = p1.x - p2.x;
6     double c1 = a1 * p1.x + b1 * p1.y;
7     double a2 = p4.y - p3.y, b2 = p3.x - p4.x;
8     double c2 = a2 * p3.x + b2 * p3.y;
9     double det = a1 * b2 - a2 * b1;
10    if (fabs(det) < 1e-9) return false; // rectas
11    // paralelas o coincidentes
12    out.x = (b2 * c1 - b1 * c2) / det;
13    out.y = (a1 * c2 - a2 * c1) / det;
14    return true;
15 }

```

9.6 Distancia Punto-Recta y Punto-Segmento

Complejidad de Tiempo: $\mathcal{O}(1)$

La distancia a la **recta** usa la proyección normal (magnitud del producto cruz normalizada). La distancia al **segmento** requiere primero verificar si la proyección cae dentro del segmento; si no, la distancia mínima es hacia uno de sus extremos.

```

1 double dist_point_line(const Point& p, const Point& A,
2     const Point& B) {
3     return fabs(cross(A, B, p)) / dist(A, B);
4 }
5 double dist_point_segment(const Point& p, const Point& A,
6     const Point& B) {
7     if (A.x == B.x && A.y == B.y) return dist(p, A); //
8     // segmento degenerado
9     double t = dot(A, B, p) / (dist(A, B) * dist(A, B));
10    // parametro de proyeccion
11    if (t < 0) return dist(p, A); // la proyeccion
12    // cae antes de A
13    if (t > 1) return dist(p, B); // la proyeccion
14    // cae despues de B
15    Point proj = {A.x + t * (B.x - A.x), A.y + t * (B.y -
16    A.y)};
17    return dist(p, proj);
18 }

```

9.7 Círculos: Pertenencia e Intersección

Complejidad de Tiempo: $\mathcal{O}(1)$

Verifica si un punto está dentro de un círculo y calcula los (hasta 2) puntos de intersección entre dos circunferencias, usando la distancia entre centros para descartar los casos sin solución (muy separados o uno contenido en el otro).

```

1 struct Circle { Point c; double r; };
2 bool point_in_circle(const Point& p, const Circle& circ) {
3     return dist(p, circ.c) <= circ.r + 1e-9;
4 }
5 // Retorna 0, 1 o 2 puntos de interseccion entre dos
6 // circunferencias
7 vector<Point> circle_circle_intersection(const Circle& c1,
8     const Circle& c2) {
9     double d = dist(c1.c, c2.c);
10    if (d > c1.r + c2.r + 1e-9 || d < fabs(c1.r - c2.r) -
11    1e-9 || d < 1e-9)
12        return {}; // sin interseccion (muy lejos, uno
13        // dentro del otro, o concentricos)
14    double a = (c1.r*c1.r - c2.r*c2.r + d*d) / (2*d);
15    double h2 = c1.r*c1.r - a*a;
16    double h = (h2 > 0) ? sqrt(h2) : 0;
17    Point p2 = { c1.c.x + a*(c2.c.x-c1.c.x)/d, c1.c.y + a
18    *(c2.c.y-c1.c.y)/d };
19    Point offset = { -(c2.c.y-c1.c.y) * (h/d), (c2.c.x-c1.
20    c.x) * (h/d) };
21    Point i1 = { p2.x + offset.x, p2.y + offset.y };

```

```

16 Point i2 = { p2.x - offset.x, p2.y - offset.y };
17 if (h < 1e-9) return {i1}; // tangentes: un solo punto
18 return {i1, i2};
19 }

```

9.8 Par de Puntos Más Cercano (Closest Pair)

Complejidad de Tiempo: $\mathcal{O}(N \log N)$ mediante divide y vencerás.

Complejidad de Espacio: $\mathcal{O}(N)$

Encuentra la distancia mínima entre dos puntos de un conjunto. Divide el conjunto ordenado por x en dos mitades, resuelve recursivamente y combina revisando solo una franja estrecha alrededor de la línea divisoria.

```

1 double closest_pair_rec(vector<Point>& pts, int l, int r)
2 {
3     if (r - l <= 3) {
4         double best = 1e18;
5         for (int i = l; i < r; i++)
6             for (int j = i+1; j < r; j++) best = min(best,
7                 dist(pts[i], pts[j]));
8         sort(pts.begin()+l, pts.begin()+r, [](const Point&
9         a, const Point& b){return a.y < b.y;});
10        return best;
11    }
12    int mid = (l + r) / 2;
13    double midX = pts[mid].x;
14    double d = min(closest_pair_rec(pts, l, mid),
15        closest_pair_rec(pts, mid, r));
16    inplace_merge(pts.begin()+l, pts.begin()+mid, pts.
17    begin()+r,
18        [](const Point& a, const Point& b){return a
19        .y < b.y;});
20    vector<Point> strip;
21    for (int i = l; i < r; i++)
22        if (fabs(pts[i].x - midX) < d) strip.push_back(pts
23        [i]);
24    for (int i = 0; i < (int)strip.size(); i++)
25        for (int j = i+1; j < (int)strip.size() && (strip[
26        j].y - strip[i].y) < d; j++)
27            d = min(d, dist(strip[i], strip[j]));
28    return d;
29 }
30 double closest_pair(vector<Point> pts) {
31     sort(pts.begin(), pts.end(), [](const Point& a, const
32     Point& b){return a.x < b.x;});
33     return closest_pair_rec(pts, 0, pts.size());
34 }

```

9.9 Rotating Calipers: Diámetro de un Conjunto de Puntos

Complejidad de Tiempo: $\mathcal{O}(N \log N)$ (dominado por el cálculo del convex hull; el barrido de calipers en sí es $\mathcal{O}(N)$). Calcula la distancia máxima entre dos puntos del conjunto. Se apoya en que el par más lejano siempre son dos vértices del *convex hull*, y recorre pares de "puntas" opuestas girando alrededor del polígono.

```

1 double poly_diameter(vector<Point> pts) {
2     vector<Point> hull = convex_hull(pts); // definido en
3     // la seccion de Convex Hull
4     int n = hull.size();
5     if (n == 1) return 0;
6     if (n == 2) return dist(hull[0], hull[1]);
7     double best = 0;
8     int j = 1;
9     for (int i = 0; i < n; i++) {
10        int ni = (i + 1) % n;
11        while (fabs(cross(hull[i], hull[ni], hull[(j+1)%n
12        ])) >

```

```

11     fabs(cross(hull[i], hull[ni], hull[j]))) {
12         j = (j + 1) % n;
13     }
14     best = max({best, dist(hull[i], hull[j]), dist(
15         hull[ni], hull[j])});
16     return best;
17 }

```

9.10 Ordenamiento Angular (Radial Sort) y Convexidad

Complejidad de Tiempo: $\mathcal{O}(N \log N)$ para el ordenamiento; $\mathcal{O}(N)$ para verificar convexidad.

Ordenar puntos por ángulo alrededor de un pivote es la base de algoritmos como Graham Scan. También se muestra cómo verificar si un polígono ya dado es convexo revisando que todos los giros consecutivos tengan el mismo signo.

```

1 // Ordena 'pts' por angulo polar alrededor de 'pivot' (sin
  // usar atan2, mas preciso)
2 void sort_by_angle(vector<Point>& pts, const Point& pivot)
3 {
4     sort(pts.begin(), pts.end(), [&](const Point& a, const
5         Point& b) {
6         bool halfA = (a.y > pivot.y) || (a.y == pivot.y &&
7             a.x > pivot.x);
8         bool halfB = (b.y > pivot.y) || (b.y == pivot.y &&
9             b.x > pivot.x);
10        if (halfA != halfB) return halfA > halfB; // parte
11        superior primero
12        double c = cross(pivot, a, b);
13        if (fabs(c) > 1e-9) return c > 0;
14        return dist(pivot, a) < dist(pivot, b); //
15        desempate: mas cercano primero
16    });
17 }
18 bool is_convex_polygon(const vector<Point>& poly) {
19     int n = poly.size();
20     if (n < 3) return false;
21     bool gotPositive = false, gotNegative = false;
22     for (int i = 0; i < n; i++) {
23         double c = cross(poly[i], poly[(i+1)%n], poly[(i
24             +2)%n]);
25         if (c > 1e-9) gotPositive = true;
26         if (c < -1e-9) gotNegative = true;
27         if (gotPositive && gotNegative) return false; //
28         cambia de signo: no convexo
29     }
30     return true;
31 }

```

9.11 Interpolación de Hermite (Cúbica)

Complejidad de Tiempo: $\mathcal{O}(1)$ por evaluación de punto.

Construye una curva suave que pasa **exactamente** por un conjunto de puntos dados, especificando además la tangente (dirección/velocidad) en cada uno. A diferencia de interpolar con un polinomio único (que oscila mucho con muchos puntos, fenómeno de Runge), Hermite trabaja *tramo por tramo* entre puntos consecutivos, garantizando continuidad de posición y primera derivada.

```

1 // Funciones base de Hermite cubico (Point definido en la
  // seccion de Primitivas)
2 double h00(double t) { return 2*t*t*t - 3*t*t + 1; }
3 double h10(double t) { return t*t*t - 2*t*t + t; }
4 double h01(double t) { return -2*t*t*t + 3*t*t; }
5 double h11(double t) { return t*t*t - t*t; }
6 // Evalua el punto en t in [0,1] entre p0 y p1, dadas las
  // tangentes m0 y m1

```

```

7 Point hermite_interp(const Point& p0, const Point& m0,
8     const Point& p1, const Point& m1,
9     double t) {
10     double H00 = h00(t), H10 = h10(t), H01 = h01(t), H11 =
11         h11(t);
12     return {
13         H00*p0.x + H10*m0.x + H01*p1.x + H11*m1.x,
14         H00*p0.y + H10*m0.y + H01*p1.y + H11*m1.y
15     };
16 }

```

Catmull-Rom: Tangentes Automáticas

Cuando no se dan tangentes explícitas, **Catmull-Rom** las calcula automáticamente a partir de los puntos vecinos, produciendo una curva suave que pasa por todos los puntos dados sin necesidad de configurar nada más. Es la forma más práctica de usar Hermite en la mayoría de los problemas (animación de rutas, suavizado de trayectorias).

```

1 // Genera tangentes automaticas: tangente_i = (p_{i+1} -
  // p_{i-1}) / 2
2 vector<Point> catmull_rom_tangents(const vector<Point>&
3     pts) {
4     int n = pts.size();
5     vector<Point> tan(n);
6     for (int i = 0; i < n; i++) {
7         Point prev = (i == 0) ? pts[i] : pts[i-1];
8         Point next = (i == n-1) ? pts[i] : pts[i+1];
9         tan[i] = { (next.x - prev.x) / 2.0, (next.y - prev
10             .y) / 2.0 };
11     }
12     return tan;
13 }
14 // Evalua un punto en la curva Catmull-Rom completa, con u
  // en el rango [0, n-1]
15 // (u = 2.3 significa "30% del camino entre el punto 2 y
  // el punto 3")
16 Point catmull_rom_eval(const vector<Point>& pts, const
17     vector<Point>& tan, double u) {
18     int i = min((int)u, (int)pts.size() - 2);
19     double t = u - i;
20     return hermite_interp(pts[i], tan[i], pts[i+1], tan[i
21         +1], t);
22 }

```

9.12 Spline Cúbico Natural

Complejidad de Tiempo:

- **Construcción:** $\mathcal{O}(N)$ (resuelve un sistema tridiagonal)
- **Evaluación:** $\mathcal{O}(\log N)$ usando búsqueda binaria para ubicar el tramo (ver capítulo de *Lower Bound* y *Upper Bound*)

Complejidad de Espacio: $\mathcal{O}(N)$

A diferencia de Hermite (que requiere tangentes dadas o estimadas), el spline cúbico natural resuelve un sistema de ecuaciones para que la curva tenga **segunda derivada continua** en todos los puntos, y además fuerza segunda derivada = 0 en los extremos (condición "natural"). Requiere que los puntos x_i estén **ordenados y sean distintos** (es una función $y = f(x)$, no una curva paramétrica general).

```

1 struct CubicSpline {
2     vector<double> x, a, b, c, d;
3     int n;
4     // xs debe estar ordenado estrictamente creciente
5     CubicSpline(const vector<double>& xs, const vector<
6         double>& ys) {
7         n = xs.size() - 1;
8         x = xs; a = ys;
9     }
10 }

```

```

8      b.assign(n + 1, 0); c.assign(n + 1, 0); d.assign(n
+ 1, 0);
9      vector<double> h(n), alpha(n + 1, 0);
10     for (int i = 0; i < n; i++) h[i] = x[i+1] - x[i];
11     for (int i = 1; i < n; i++)
12         alpha[i] = 3.0*(a[i+1]-a[i])/h[i] - 3.0*(a[i]-
a[i-1])/h[i-1];
13     // Resuelve el sistema tridiagonal para los
coeficientes 'c' (segundas derivadas/6)
14     vector<double> l(n + 1), mu(n + 1), z(n + 1);
15     l[0] = 1; mu[0] = 0; z[0] = 0;
16     for (int i = 1; i < n; i++) {
17         l[i] = 2*(x[i+1] - x[i-1]) - h[i-1]*mu[i-1];
18         mu[i] = h[i] / l[i];
19         z[i] = (alpha[i] - h[i-1]*z[i-1]) / l[i];
20     }
21     l[n] = 1; z[n] = 0; c[n] = 0;
22     for (int j = n - 1; j >= 0; j--) {
23         c[j] = z[j] - mu[j]*c[j+1];
24         b[j] = (a[j+1]-a[j])/h[j] - h[j]*(c[j+1] + 2*c
[j])/3.0;
25         d[j] = (c[j+1] - c[j]) / (3.0*h[j]);
26     }
27 }
28 // Evalua el spline en xq (busqueda binaria del tramo
correspondiente)
29 double eval(double xq) {
30     int i = upper_bound(x.begin(), x.end(), xq) - x.
begin() - 1;
31     i = max(0, min(i, n - 1));
32     double dx = xq - x[i];
33     return a[i] + b[i]*dx + c[i]*dx*dx + d[i]*dx*dx*dx
; // a + b*dx + c*dx^2 + d*dx^3
34 }
35 };

```

10. Fórmulas Matemáticas Esenciales (Implementadas)

Referencia rápida de fórmulas que aparecen con frecuencia en el planteamiento de problemas, ya traducidas a código C++ listo para copiar y adaptar.

10.1 Sumas y Series

Complejidad de Tiempo: $\mathcal{O}(1)$ para todas las funciones de esta sección (fórmulas cerradas, sin iterar).

```
1 // Suma de los primeros n naturales: 1+2+...+n = n(n+1)/2
2 ll suma_naturales(ll n) { return n * (n + 1) / 2; }
3
4 // Suma de los primeros n numeros impares: 1+3+5+...+(2n-1) = n^2
5 ll suma_impares(ll n) { return n * n; }
6
7 // Suma de cuadrados: 1^2+2^2+...+n^2 = n(n+1)(2n+1)/6
8 ll suma_cuadrados(ll n) { return n * (n + 1) * (2 * n + 1) / 6; }
9
10 // Suma de cubos: 1^3+2^3+...+n^3 = (n(n+1)/2)^2
11 ll suma_cubos(ll n) { ll s = suma_naturales(n); return s * s; }
12
13 // Suma de una progresion aritmetica con n terminos, primer termino a1, razon d
14 ll suma_progresion_aritmetica(ll a1, ll d, ll n) {
15     ll an = a1 + (n - 1) * d; // ultimo termino
16     return n * (a1 + an) / 2;
17 }
18
19 // Suma de una progresion geometrica finita: a1 + a1*r + ... + a1*r^(n-1), r != 1
20 // (usar modpow/inv_mod si se necesita modulo, ver capitulo de Matematicas Teoricas)
21 double suma_geometrica(double a1, double r, ll n) {
22     if (r == 1) return a1 * n;
23     return a1 * (pow(r, n) - 1) / (r - 1);
24 }
25
26 // Suma geometrica infinita, |r| < 1
27 double suma_geometrica_infinita(double a1, double r) {
28     return a1 / (1 - r);
29 }
30
31 // Aproximacion de la serie armonica: 1 + 1/2 + 1/3 + ... + 1/n
32 // Util para estimar complejidad O(n log n) de cribas/ divisores sin sumar termino a termino
33 const double EULER_MASCHERONI = 0.5772156649;
34 double serie_armonica_aprox(ll n) {
35     return log((double)n) + EULER_MASCHERONI;
36 }
```

10.2 Combinatoria

Complejidad de Tiempo: $\mathcal{O}(1)$ con nCr precalculado (ver fact/inv_fact en el capítulo de Matemáticas Teóricas); $\mathcal{O}(N)$ para Catalan/derangements si se calculan en tabla.

```
1 // Permutaciones sin repeticion: P(n,r) = n! / (n-r)!
2 // Requiere fact[] e inv_fact[] precalculados (ver Combinatoria en Matematicas Teoricas)
3 ll nPr(int n, int r) {
4     if (r < 0 || r > n) return 0;
5     return fact[n] * inv_fact[n - r] % MOD;
6 }
7 // Combinaciones: ya definido como nCr(n, r) en el capitulo de Matematicas Teoricas
8 // (se repite aqui la firma como referencia): ll nCr(int n, int r);
9
10 // Total de subconjuntos de un conjunto de n elementos: 2^n
```

```
11 ll total_subconjuntos(int n) { return modpow(2, n, MOD); }
12
13 // Combinaciones con repeticion (multiconjuntos de tamaño r tomados de n tipos)
14 ll combinaciones_con_repeticion(int n, int r) { return nCr(n + r - 1, r); }
15
16 // Permutaciones con repeticion: n! / (n1! * n2! * ... * nk!)
17 ll permutaciones_con_repeticion(int n, const vector<int>& grupos) {
18     ll res = fact[n];
19     for (int ni : grupos) res = res * inv_fact[ni] % MOD;
20     return res;
21 }
22
23 // Identidad de Vandermonde: C(m+n, r) = sum_{k=0}^r C(m, k) * C(n, r-k)
24 // Sirve como checksum/derivacion; rara vez se usa directamente en codigo, pero se implementa por si se necesita validar o calcular de forma indirecta
25 ll vandermonde(int m, int n, int r) {
26     ll res = 0;
27     for (int k = 0; k <= r; k++) res = (res + nCr(m, k) * nCr(n, r - k)) % MOD;
28     return res;
29 }
30
31 // Identidad del palo de hockey: sum_{i=r}^n C(i, r) = C(n+1, r+1)
32 // (evita sumar n-r terminos; calcular directo con nCr(n+1, r+1))
33 ll hockey_stick(int n, int r) { return nCr(n + 1, r + 1); }
34
35 // Numero de derangements (permutaciones sin puntos fijos), via recurrencia
36 // D_n = (n-1) * (D_{n-1} + D_{n-2}), D_0 = 1, D_1 = 0
37 vector<ll> tabla_derangements(int n) {
38     vector<ll> D(n + 1);
39     D[0] = 1; if (n >= 1) D[1] = 0;
40     for (int i = 2; i <= n; i++)
41         D[i] = (ll)(i - 1) * ((D[i-1] + D[i-2]) % MOD) % MOD;
42     return D;
43 }
44
45 // Numeros de Catalan: C_n = (1/(n+1)) * C(2n, n) = sum_{i=0}^{n-1} C_i * C_{n-1-i}
46 vector<ll> tabla_catalan(int n) {
47     vector<ll> C(n + 1, 0);
48     C[0] = 1;
49     for (int i = 1; i <= n; i++)
50         for (int j = 0; j < i; j++)
51             C[i] = (C[i] + C[j] * C[i - 1 - j]) % MOD;
52     return C;
53 }
54
55 // Version O(1) por consulta usando factoriales precalculados:
56 ll catalan_directo(int n) {
57     return (nCr(2 * n, n) - nCr(2 * n, n + 1) % MOD + MOD) % MOD;
58 }
```

10.3 Geometría

Complejidad de Tiempo: $\mathcal{O}(1)$ por fórmula (ver también el capítulo de Geometría Computacional para Point, cross, dist, convex hull, etc.).

```
1 // Area de triangulo con formula de Heron (lados a, b, c)
2 double area_triangulo_heron(double a, double b, double c) {
3     double s = (a + b + c) / 2.0;
4     return sqrt(s * (s - a) * (s - b) * (s - c));
5 }
6
```

```

7 // Area de triangulo por coordenadas (shoelace de 3 puntos
8 double area_triangulo_coords(double x1, double y1, double
  x2, double y2, double x3, double y3) {
9     return fabs(x1*(y2-y3) + x2*(y3-y1) + x3*(y1-y2)) /
10    2.0;
11 }
12 // Area de trapezio (bases b1, b2, altura h)
13 double area_trapezio(double b1, double b2, double h) {
14     return (b1 + b2) * h / 2.0; }
15
16 // Area de poligono regular de n lados de longitud l
17 double area_poligono_regular(int n, double l) {
18     return (n * l * l) / (4.0 * tan(M_PI / n));
19 }
20 // Teorema de Pitagoras: dado a y b, obtener hipotenusa; o
21 // despejar un cateto
22 double hipotenusa(double a, double b) { return sqrt(a*a +
  b*b); }
23 double cateto(double hip, double otroCateto) { return sqrt
  (hip*hip - otroCateto*otroCateto); }
24
25 // Ley de cosenos: c^2 = a^2 + b^2 - 2ab*cos(C), con C en
26 // radianes
27 double ley_cosenos(double a, double b, double anguloC_rad)
28 {
29     return sqrt(a*a + b*b - 2*a*b*cos(anguloC_rad));
30 }
31 // Despejar el angulo C conocidos los 3 lados
32 double angulo_desde_lados(double a, double b, double c) {
33     return acos((a*a + b*b - c*c) / (2*a*b)); // en
34     radianes
35 }
36 // Ley de senos: a/sin(A) = b/sin(B) = c/sin(C). Dado un
37 // lado y su angulo opuesto,
38 // mas otro angulo, obtener el lado faltante.
39 double ley_senos_lado(double ladoConocido, double
  anguloOpuestoConocido_rad, double
  anguloOpuestoBuscado_rad) {
40     return ladoConocido * sin(anguloOpuestoBuscado_rad) /
41     sin(anguloOpuestoConocido_rad);
42 }
43
44 // Distancia entre dos puntos (ver tambien dist(Point,
45 // Point) en Geometria Computacional)
46 double distancia_2d(double x1, double y1, double x2,
  double y2) {
47     return sqrt((x2-x1)*(x2-x1) + (y2-y1)*(y2-y1));
48 }
49
50 // Circulo: perimetro y area
51 double circulo_perimetro(double r) { return 2.0 * M_PI * r
  ; }
52 double circulo_area(double r) { return M_PI * r * r; }
53
54 // Esfera: volumen y area superficial
55 double esfera_volumen(double r) { return (4.0/3.0) * M_PI
  * r * r * r; }
56 double esfera_area(double r) { return 4.0 * M_PI * r * r;
  }
57
58 // Cilindro: volumen y area total (2 tapas + lateral)
59 double cilindro_volumen(double r, double h) { return M_PI
  * r * r * h; }
60 double cilindro_area_total(double r, double h) { return 2*
  M_PI*r*h + 2*M_PI*r*r; }
61
62 // Cono: volumen y area lateral (g = generatriz = sqrt(r^2
63 // + h^2))
64 double cono_volumen(double r, double h) { return (M_PI * r
  * r * h) / 3.0; }
65 double cono_area_lateral(double r, double h) {
66     double g = sqrt(r*r + h*h);
67     return M_PI * r * g;
68 }
69
70 // Teorema de Pick: Area = I + B/2 - 1
71 // I = puntos enteros interiores, B = puntos enteros en el
72 // borde
73 double pick_area(ll puntosInteriores, ll puntosBorde) {
74     return puntosInteriores + puntosBorde / 2.0 - 1.0;
75 }
76 // Puntos enteros estrictamente entre dos vertices de
77 // coordenadas enteras (para calcular B)

```

```

69 ll puntos_enteros_en_segmento(ll x1, ll y1, ll x2, ll y2)
70 {
71     return __gcd(llabs(x2 - x1), llabs(y2 - y1)) - 1;
72 }
73 // Dado el area (ej. por shoelace) y el borde B, despejar
74 // los puntos interiores I
75 ll pick_puntos_interiores(double area, ll puntosBorde) {
76     return (ll)llround(area - puntosBorde / 2.0 + 1.0);
77 }

```

10.4 Probabilidad y Recurrencias

Complejidad de Tiempo: $\mathcal{O}(1)$ para las fórmulas puntuales; $\mathcal{O}(\log n)$ para Fibonacci vía exponenciación de matrices.

```

1 // Esperanza de la suma:  $E[X+Y] = E[X] + E[Y]$  (SIEMPRE se
2 // cumple, sin importar independencia)
3 double esperanza_suma(const vector<double>&
4 // esperanzasIndividuales) {
5     double total = 0;
6     for (double e : esperanzasIndividuales) total += e;
7     return total;
8 }
9
10 // Varianza:  $Var(X) = E[X^2] - (E[X])^2$ 
11 double varianza(const vector<double>& valores, const
12 // vector<double>& probabilidades) {
13     double EX = 0, EX2 = 0;
14     for (int i = 0; i < (int)valores.size(); i++) {
15         EX += valores[i] * probabilidades[i];
16         EX2 += valores[i] * valores[i] * probabilidades[i];
17     }
18     return EX2 - EX * EX;
19 }
20
21 // Teorema de Bayes:  $P(A|B) = P(B|A) * P(A) / P(B)$ 
22 double bayes(double pB_dado_A, double pA, double pB) {
23     return (pB_dado_A * pA) / pB;
24 }
25
26 // Probabilidad total:  $P(A) = \sum_i P(A|B_i) * P(B_i)$ 
27 // pA_dado_B[i] y pB[i] son vectores paralelos, uno por
28 // cada evento B_i del particionamiento
29 double probabilidad_total(const vector<double>& pA_dado_B,
30 // const vector<double>& pB) {
31     double res = 0;
32     for (int i = 0; i < (int)pA_dado_B.size(); i++) res +=
33     pA_dado_B[i] * pB[i];
34     return res;
35 }
36
37 // Inclusion-exclusion general para n conjuntos, dado |
38 // interseccion de cualquier subconjunto S|
39 // como funcion interseccion(S) donde S es una mascara de
40 // bits (que conjuntos intervienen)
41 // |Union| = sum sobre todas las submascaras no vacias,
42 // con signo (-1)^(|S|+1)
43 template <typename F>
44 ll inclusion_exclusion(int n, F interseccion) {
45     ll total = 0;
46     for (int mask = 1; mask < (1 << n); mask++) {
47         int bits = __builtin_popcount(mask);
48         ll inter = interseccion(mask);
49         total += (bits % 2 == 1) ? inter : -inter;
50     }
51     return total;
52 }
53
54 // Caso base con 2 conjuntos:  $|A \cup B| = |A| + |B| - |A \cap B|$ 
55 ll union_dos_conjuntos(ll tamA, ll tamB, ll
  tamInterseccion) {
56     return tamA + tamB - tamInterseccion;
57 }
58
59 // Fibonacci: formula cerrada (Binet) -- ojo con precision
60 // para n grande, mejor usar matrices
61 double fibonacci_binet(int n) {
62     const double phi = (1.0 + sqrt(5.0)) / 2.0;
63     const double psi = (1.0 - sqrt(5.0)) / 2.0;
64     return round((pow(phi, n) - pow(psi, n)) / sqrt(5.0));
65 }

```

```

56 // Fibonacci mod M en  $O(\log n)$  via exponenciacion de
    matrices 2x2
57 //  $[[F(n+1), F(n)], [F(n), F(n-1)]] = [[1,1],[1,0]]^n$ 
58 struct Mat2 { ll a, b, c, d; };
59 Mat2 matmul(const Mat2& X, const Mat2& Y, ll mod) {
60     return {
61         (X.a*Y.a + X.b*Y.c) % mod, (X.a*Y.b + X.b*Y.d) %
        mod,
62         (X.c*Y.a + X.d*Y.c) % mod, (X.c*Y.b + X.d*Y.d) %
        mod
63     };
64 }
65 ll fibonacci_matriz(ll n, ll mod) {
66     if (n == 0) return 0;
67     Mat2 result = {1, 0, 0, 1}; // identidad
68     Mat2 base = {1, 1, 1, 0};
69     ll e = n;
70     while (e > 0) {
71         if (e & 1) result = matmul(result, base, mod);
72         base = matmul(base, base, mod);
73         e >>= 1;
74     }
75     return result.b; // F(n) queda en la posicion [0][1]
76 }

```

10.5 Teorema Maestro (guía de referencia, no se codea)

Para recurrencias tipo $T(n) = a \cdot T(n/b) + f(n)$ con $f(n) = O(n^d)$, esto no se implementa (es una herramienta de análisis en papel), pero conviene tenerlo a mano para estimar la complejidad de un divide y vencerás antes de programarlo:

```

1 // Guia rapida (comparar d contra log_b(a)):
2 // d < log_b(a) -> T(n) =  $O(n^{\log_b a})$ 
3 // d == log_b(a) -> T(n) =  $O(n^d \log n)$ 
4 // d > log_b(a) -> T(n) =  $O(n^d)$ 
5 // Ejemplo: Merge Sort -> T(n) =  $2T(n/2) + O(n)$  => a=2,
    b=2, d=1, log_2(2)=1 =>  $O(n \log n)$ 
6 // Ejemplo: Karatsuba -> T(n) =  $3T(n/2) + O(n)$  => a=3,
    b=2, d=1, log_2(3)=1.58 =>  $O(n^{1.58})$ 
7 double log_b(double a, double b) { return log(a) / log(b);
    } // ayuda a comparar d vs log_b(a)

```

11. Casos de Estudio: Problemas Resueltos

11.1 Matrioshka

```
1 #include <bits/stdc++.h>
2 #include <sstream>
3 #include <string>
4 using namespace std;
5 int main() {
6     string a;
7     while(getline(cin, a)) {
8         stringstream ss(a);
9         stack<int> mat;
10        stack<int> sumcajas;
11        bool rot = false;
12        int c;
13        while(ss >> c) {
14            if(c < 0) {
15                mat.push(-c);
16                sumcajas.push(0);
17            }
18            else if(!rot) {
19                if(mat.empty() || mat.top() != c) {
20                    rot = true;
21                }
22                else {
23                    mat.pop();
24                    int hijos = sumcajas.top();
25                    sumcajas.pop();
26                    if(hijos >= c) {
27                        rot = true;
28                    }
29                    else if(!sumcajas.empty()) {
30                        int padre = sumcajas.top();
31                        sumcajas.pop();
32                        sumcajas.push(padre + c);
33                    }
34                }
35            }
36        }
37        if(!mat.empty()) rot = true;
38        if(rot) cout << ":-( Try again." << endl;
39        else cout << ":-) Matrioshka!" << endl;
40    }
41 }
```